



上海交通大学学位论文

面向 3C 工业生产线的双臂机器人视觉-语言-动作模型

姓名：杨皓然 521111910083

吴律 522370910167

吴毅昕 522370910185

程天纵 522370910227

顾益铭 522370910228

导师：班雨桐

指导教师：黄佩森

学院：浦江国际学院

专业名称：电子与计算机工程

申请学位层次：学士

2026 年 8 月

A Dissertation Submitted to

Shanghai Jiao Tong University for the Degree of Bachelor

A VISION-LANGUAGE-ACTION MODEL FOR

DUAL-ARM ROBOTS IN 3C INDUSTRIAL

PRODUCTION LINES

Name:	Yang Haoran	521111910083
	Wu Lv	522370910167
	Wu Yixin	522370910185
	Cheng Tianzong	522370910227
	Gu Yiming	522370910228
Mentor:	Prof. Yutong Ban	
Instructor:	Prof. Peisen Huang	
College:	Global College	

Global College

Shanghai Jiao Tong University

Shanghai, P.R. China

August, 2026

上海交通大学

学位论文原创性声明

本人郑重声明：所呈交的学位论文，是本人在导师的指导下，独立进行研究工作所取得的成果。除文中已经注明引用的内容外，本论文不包含任何其他个人或集体已经发表或撰写过的作品成果。对本文的研究做出重要贡献的个人和集体，已在文中以适当方式予以致谢。若在论文撰写过程中使用了人工智能工具，本人已遵循《上海交通大学关于在教育教学中使用 AI 的规范》，确保人工智能生成内容的应用场景、引用范围及标注方式均符合规定，并杜绝学术不端行为。本人完全知晓本声明的法律后果由本人承担。

学位论文作者签名：

日期： 年 月 日

上海交通大学

学位论文使用授权书

本人同意学校保留并向国家有关部门或机构送交论文的复印件和电子版，允许论文被查阅和借阅。

本学位论文属于：

☐ 公开论文

☐ 内部论文，保密 ☐ 1 年 / ☐ 2 年 / ☐ 3 年，过保密期后适用本授权书。

☐ 秘密论文，保密 ____ 年（不超过 10 年），过保密期后适用本授权书。

☐ 机密论文，保密 ____ 年（不超过 20 年），过保密期后适用本授权书。

（请在以上方框内选择打“√”）

学位论文作者签名：

日期： 年 月 日

指导教师签名：

日期： 年 月 日

摘 要

现代 3C 电子装配线需要处理小型元器件、高产品多样性和富接触操作，这些任务仍难以通过传统固定程序机器人实现自动化。本项目针对这一缺口，面向双臂工业操作开发视觉-语言-动作（Vision-Language-Action, VLA）系统。系统运行于双臂 Franka 平台，并完成了三个代表性任务：将带手柄的圆环套放到圆柱罐上、将两脚插头插入两孔插座，以及从主板上取下内存条并放入黄色托盘。

最终部署系统采用统一训练的 $\pi_{0.5}$ 检查点，接收三个同步 RGB 图像、16 维双臂机器人状态和任务语言指令，并在闭环执行过程中输出双臂及夹爪动作。三个任务均由同一个检查点根据对应语言指令执行，无需加载任务特定模型。该系统已在真实双 Franka 工作站上完成语言条件多任务双臂控制验证。

最终实机验证采用 65,000 步检查点，对每项任务进行了 25 次试验，共计 75 次。圆环任务、插头插入任务和内存条任务的成功率分别为 84%、52% 和 100%，总体成功率为 78.7%。独立的 25 次试验还得到规划成功率 92%、元件检测率 96%、失败检测率 92% 和恢复成功率 92%；按所采用的记录规则，位姿误差小于 20 mm。六项工程指标的项目级点估计或操作性测量均达到原定阈值，但插头插入任务未能单独达到 70% 的任务执行目标，表明接触丰富的精细对准与插入仍是当前系统的主要限制。

关键词：双臂机器人，视觉-语言-动作模型，工业自动化

ABSTRACT

Modern 3C electronics assembly lines deal with small components, high product variety, and contact-rich operations that remain difficult to automate with conventional fixed-program robots. Our project targets this gap by developing a Vision-Language-Action (VLA) system for bimanual industrial manipulation. The system runs on a dual-arm Franka platform and performs three representative tasks: placing a handled ring over a can, inserting a two-pin plug into a two-hole socket, and removing a memory module from a motherboard and placing it in a yellow tray.

The final deployed system uses one jointly trained $\pi_{0.5}$ checkpoint. It receives three synchronized RGB images, a 16-dimensional bilateral robot state, and a task-specific language instruction, and outputs bimanual robot and gripper actions during closed-loop execution. All three tasks are executed by the same checkpoint under their corresponding language instructions without loading task-specific models. The system has therefore demonstrated language-conditioned multi-task bimanual control on the physical dual-Franka workcell.

The final physical validation used the 65,000-step checkpoint and comprised 25 trials per task and 75 trials in total. The ring, plug-insertion, and memory-module tasks achieved success rates of 84%, 52%, and 100%, respectively, producing an overall success rate of 78.7%. Separate 25-experiment tests produced 92% planning success, 96% component detection, 92% failure detection, and 92% recovery; the recorded pose error was below 20 mm under the stated measurement rule. All six project-level point estimates or operational measurements met their original thresholds, although the plug-insertion task did not independently meet the 70% execution target, identifying contact-rich fine alignment and insertion as the principal limitation of the current system.

Key words: dual-arm robot, vision-language-action model, industrial automation

Contents

摘 要	I
ABSTRACT	II
Chapter 1 Introduction	1
1.1 Background and significance	1
1.2 Design problem	2
1.3 Project objectives and scope	3
1.4 Main contributions	3
1.5 Organization of this thesis	4
Chapter 2 Design specification	5
2.1 Customer requirements	5
2.2 Engineering specifications	6
2.3 Quality function deployment	6
Chapter 3 Concept generation and selection	8
3.1 Concept generation	8
3.1.1 Research landscape of imitation learning and vision-language-action models ...	8
3.1.2 Functional decomposition	14
3.1.3 Structured brainstorming	14
3.1.4 Morphological chart and generated module concepts	15
3.2 Concept selection	16
3.2.1 Selection method and QFD-derived criteria	16
3.2.2 Scoring of the five selected module concepts	17
3.2.3 Comparative evaluation and selected concept	18
Chapter 4 Final design and implementation	20
4.1 Final design and engineering changes	20
4.1.1 Design evolution from the DR3 baseline	20

4.1.2	Engineering Change Notice from DR3 to the final prototype	20
4.1.3	Overall system architecture	23
4.1.4	VLA input–output formulation and notation.....	24
4.1.5	Selected module concepts	25
4.1.6	Operational workflow	27
4.2	Engineering design analysis	28
4.2.1	Relevant engineering fundamentals	28
4.2.2	Design assumptions and constraints	28
4.2.3	FR3 parameters and bimanual kinematic suitability.....	29
4.2.4	Bimanual state and action representation	31
4.2.5	Shared workspace and role allocation	31
4.2.6	Contact, clearance, and insertion tolerance analysis.....	32
4.2.7	Custom gripper geometry and mechanical analysis	33
4.2.8	Perception working distance, field of view, and camera selection	34
4.2.9	Image format and preprocessing	34
4.2.10	GELLO channel architecture and leader–follower mapping	35
4.2.11	Data-collection pipeline	36
4.2.12	Packet synchronization and action-label alignment	36
4.2.13	Dataset construction and episode split	37
4.2.14	Dataset inclusion boundary and language-prompt contract	38
4.2.15	Training statistics and task-balanced sampling.....	39
4.2.16	$\pi_{0.5}$ base model and software configuration	40
4.2.17	Model input construction and state–language representation	40
4.2.18	Flow-matching training objective	41
4.2.19	Adaptation scope and trainable parameters	41
4.2.20	Training procedure.....	42
4.2.21	Optimizer schedule, runtime, and checkpoint package	43
4.2.22	Validation and checkpoint selection.....	44
4.2.23	Action chunking and temporal interface.....	44
4.2.24	Deployment pipeline	45
4.2.25	Control, computation, and safety boundary.....	45

4.3	Detailed system design	46
4.3.1	Workcell layout	46
4.3.2	Dual-arm robotic platform	46
4.3.3	GELLO leader assemblies.....	46
4.3.4	End-effector and mechanical components	47
4.3.5	Vision and sensing system	47
4.3.6	Demonstration data-collection system.....	47
4.3.7	Data processing pipeline	48
4.3.8	Policy-training system	48
4.3.9	Robot control and deployment system.....	48
4.3.10	Hardware–software interfaces	49
4.3.11	Task definitions and bilateral role allocation.....	50
4.3.12	System operation	50
4.3.13	Parts and components	52
4.4	Prototype manufacturing and implementation.....	52
4.4.1	Implementation scope.....	52
4.4.2	Materials and manufacturing processes	53
4.4.3	Gripper fabrication and inspection	53
4.4.4	GELLO assembly and calibration.....	54
4.4.5	Camera and fixture implementation	54
4.4.6	Software implementation.....	55
4.4.7	System integration sequence	55
4.4.8	Budget	56
4.4.9	Software, environment, and checkpoint provenance.....	57
4.4.10	Reproduction checklist for the final configuration.....	57
4.4.11	Current implementation status and remaining documentation	58
Chapter 5	Validation results	59
5.1	Validation objectives and scope	59
5.2	Validation design and engineering-specification mapping	60
5.2.1	Measurement rules and independent denominators	60
5.2.2	Observed specification-level outcomes	61

5.3	Checkpoint selection and deployment configuration	63
5.3.1	Selection criterion	63
5.3.2	Validation trajectory	63
5.3.3	Deployment hardware	64
5.4	Physical trial protocol and success definition	65
5.5	Overall physical results	66
5.5.1	Descriptive uncertainty of the observed rates	67
5.6	Task-specific results and failure analysis	68
5.6.1	Memory-module removal and tray placement	68
5.6.2	Ring placement over the can	69
5.6.3	Two-pin plug grasping and two-hole socket insertion	69
5.7	Planning, detection, pose-error, failure-detection, and recovery results	70
5.7.1	Rate-based engineering specifications	70
5.7.2	ES4 pose-error result and reporting rule	72
5.8	Cross-task interpretation	72
5.8.1	Requirements-level interpretation	73
5.9	Validation limitations	73
5.10	Validation summary	75
Chapter 6	Discussion and conclusions	76
6.1	Discussion	76
6.2	Conclusions	78
Chapter 7	Future Work	79
7.1	Improve contact-rich task performance	79
7.2	Develop autonomous failure detection and recovery	80
7.3	Expand data coverage and model evaluation	80
7.4	Improve language conditioning and task generalization	81
7.5	Close mechanical, sensing, and timing documentation gaps	81
7.6	Prepare for a more reproducible and deployable system	82
7.7	Recommended continuation sequence	83

References.....	84
Appendix I. Documentation of Generated Concepts.....	86
Appendix II. Hardware Specifications and Parts Records	89
Acknowledgements	98

Chapter 1 Introduction

1.1 Background and significance

The 3C industry—computer, communication, and consumer electronics—increasingly demands flexible robotic systems that can cope with short product life cycles, diverse part geometries, small components, and frequent line changeovers. Traditional hard-coded automation approaches are effective for stable, high-volume production, but they exhibit significant limitations in high-mix assembly scenarios. Each product variant typically requires reprogramming, fixture redesign, and system reconfiguration, which increases development and maintenance costs. In contrast, human operators can rapidly understand new tasks from natural-language instructions and adapt to new operating conditions with limited prior programming. This contrast highlights the importance of advancing industrial robotic systems toward more human-aligned capabilities, especially semantic understanding of instructions and data-efficient adaptation under limited demonstrations.

Recent advances in Vision-Language-Action (VLA) models offer a promising route toward this goal. By jointly learning from visual observations, language instructions, robot proprioceptive states, and action data, VLA models can in principle generalize across tasks without requiring separate hand-engineered programs for every product variant. Landmark systems, including RT-2, OpenVLA, RDT-1B, and the π series, demonstrate the potential of transferring semantic knowledge from large vision-language models to physical robot control^[1-4]. Earlier work on visual and language-conditioned manipulation also established complementary design patterns: Transporter Networks use spatially grounded pick-and-place representations, CLIPort combines semantic and spatial pathways, and PerAct couples language goals to structured visual observations^[5-7]. In parallel, BridgeData V2, Open X-Embodiment, DROID, and LIBERO have made dataset scale, embodiment diversity, and reproducible task evaluation central considerations in robot-learning system design^[8-11].

Despite this progress, adapting VLA-style systems to industrial assembly remains a system-level engineering problem. Many published demonstrations emphasize tabletop manipulation, single-arm control, or short-horizon tasks with relatively forgiving tolerances. In contrast, 3C assembly requires reliable perception of small components, accurate contact-rich

manipulation, synchronized sensing, safe robot execution, and repeatable validation under bounded physical conditions. These requirements motivate a complete manipulation pipeline that links hardware selection, teleoperation, data collection, model training, deployment, and quantitative evaluation.

1.2 Design problem

This project develops a dual-arm industrial manipulation platform that couples camera observations, natural-language task instructions, and robot proprioceptive state to learned bilateral actions. The workcell consists of two Franka Research 3 (FR3) robot arms, one fixed overhead camera, two wrist cameras, two custom grippers, and two GELLO leader devices for demonstration collection. Demonstrations are recorded as packet-associated visual observations, bilateral robot and gripper states, absolute target actions, and task instructions.

The implemented task set contains three bimanual manipulation sequences. In the first task, the left arm pushes a can to the middle while the right arm grasps a handled black ring, places it over the can, and releases it. In the second task, the right arm pushes a box containing a power socket to the middle while the left arm grasps a plug and inserts it into the socket. In the third task, the left arm pulls and stabilizes a computer motherboard while the right arm removes the memory module and places it in a yellow tray. The first task serves as a structured reposition–grasp–place benchmark, while the latter two represent constrained connector and computer-component operations.

The final controller is the LeRobot implementation of $\pi_{0.5}$, initialized from the official `pi05_base` checkpoint^[12-15]. One jointly trained checkpoint is shared by all three tasks. At deployment, the same policy receives one of the documented natural-language instructions, three RGB observations, and the current 16-dimensional bilateral state, then predicts absolute targets for both arms and both grippers. The completed implementation therefore provides language-conditioned task input, unified multi-task inference, and dual-arm action generation within the three bounded workcell tasks. It does not claim unrestricted open-world generalization, autonomous task planning, certified failure recovery, or production-line autonomy beyond the evaluated conditions.

1.3 Project objectives and scope

The project objective is to establish an end-to-end engineering pipeline for bounded bimanual industrial-manipulation experiments. This pipeline includes hardware integration, packet-associated multi-camera sensing, bilateral teleoperated demonstration collection, LeRobot Dataset v3.0 preparation, joint multi-task $\pi_{0.5}$ fine-tuning, language-conditioned deployment, and repeatable physical validation of the three scoped tasks.

The engineering acceptance targets inherited from the design-specification stage are planning success above 90%, task execution success above 70%, component detection accuracy above 90%, pose error below 20 mm, failure-detection rate above 90%, and recovery success above 70%. These values define the acceptance thresholds used by the final validation rather than assumed performance. Chapter 5 reports separate 25-experiment tests for planning, detection, pose error, failure detection, and recovery, together with 75 counted end-to-end task trials. Their denominators and operational definitions remain separate so that an intermediate result cannot be substituted for complete physical task success.

1.4 Main contributions

The main contributions of this thesis are as follows:

1. A dual-FR3 manipulation platform is specified and integrated with GELLO teleoperation, synchronized camera sensing, robot-state recording, and task-compatible end effectors and fixtures.
2. A reproducible data and learning pipeline converts 240 bilateral teleoperated episodes into three LeRobot Dataset v3.0 task datasets with episode-level training and validation splits, equal-probability task sampling, and a common absolute-action representation.
3. A single language-conditioned LeRobot $\pi_{0.5}$ checkpoint is fine-tuned and deployed for three bimanual tasks using three RGB views, a 16-dimensional bilateral state, and a 16-dimensional bilateral action interface.
4. A specification-led physical-validation framework maps ES1–ES6 to explicit operational definitions, separate denominators, task-specific terminal conditions, uncertainty estimates, failure stages, and pass/fail judgments.
5. A traceable engineering process links customer requirements and quality function de-

ployment to concept generation, module selection, implementation decisions, and validation criteria.

1.5 Organization of this thesis

The remainder of this thesis is organized as follows. Chapter 2 translates customer requirements into measurable engineering specifications and presents the quality function deployment. Chapter 3 describes functional decomposition, concept generation, and selection of the major hardware, sensing, teleoperation, end-effector, and policy modules. Chapter 4 details the final system design, data and control architecture, model-training procedure, and implementation plan. Chapter 5 presents the physical validation methods and experimental results. Chapter 6 discusses the strengths and limitations of the system and summarizes the project conclusions. Chapter 7 presents system-level and implementation-level recommendations for future development.

Chapter 2 Design specification

2.1 Customer requirements

Through discussions with the project sponsor and analysis of the target application, eight customer requirements were identified and assigned importance weights reflecting the sponsor's priorities. The requirements capture both functional goals and non-functional concerns, including robustness, safety, generalizability, cost, and reliability.

Table 2–1 Customer requirements and importance weights

No.	Customer Requirement	Weight
CR1	Complete assembly task successfully	0.18
CR2	High assembly precision	0.16
CR3	Robust to environment and disturbances	0.15
CR4	Safe and collision-free operation	0.14
CR5	Fast task execution	0.12
CR6	Easy to use and generalize	0.10
CR7	Cost-effective system	0.08
CR8	System reliable and recovery capable	0.07

The customer requirements were translated into engineering specifications through the QFD process rather than by assigning targets independently. For each customer requirement i and engineering specification j , the team assigned a relationship value $r_{ij} \in \{0, 1, 3, 9\}$ (none, weak, moderate, or strong) and calculated the unnormalized engineering priority as

$$I_j = \sum_i w_i r_{ij}, \quad (2-1)$$

where w_i is the customer-importance weight in Table 2–1. The priorities were then normalized to identify the technical characteristics that most directly protect the high-weight needs. CR1 and CR5 primarily drive ES1 and ES2; CR2 drives ES3 and ES4; CR3, CR4, and CR8 drive ES5 and ES6; and CR6–CR7 constrain the selected implementation to a usable, feasible, and maintainable route. This translation produces an auditable chain from the sponsor's language to acceptance tests and explains why the later concept search is organized around platform, teleoperation, data collection, gripper, and policy modules.

2.2 Engineering specifications

The customer requirements were converted into six measurable engineering specifications. The targets in Table 2–2 are the final acceptance thresholds for the present proof-of-concept, rather than performance claims. Each rate is calculated over independently initialized physical trials of the relevant task. A trial is counted once only; an unsuccessful recovery is not reclassified as an independent execution attempt. This convention prevents recovery behavior from artificially inflating the reported task-execution success rate.

Table 2–2 Final engineering specifications and acceptance thresholds

No.	Engineering Specification	Target	Operational Definition
ES1	Planning success rate	> 90%	Fraction of trials in which the selected task policy and initialized scene form a valid executable plan.
ES2	Task execution success rate	> 70%	Fraction of trials in which the prescribed assembly operation reaches its task-specific completion condition.
ES3	Component detection accuracy	> 90%	Fraction of evaluated observations in which the required component is correctly detected for the active task.
ES4	Pose estimation error	< 20 mm	Euclidean position error between the estimated and reference component pose.
ES5	Failure detection rate	> 90%	Fraction of induced or naturally occurring execution failures that are identified before further unsafe action is issued.
ES6	Recovery rate	> 70%	Fraction of detected failures for which the defined recovery procedure restores a valid task state.

2.3 Quality function deployment

Figure 2–1 records the resulting House of Quality. Its 9–3–1 relationship scale makes the engineering priorities interpretable rather than decorative: task completion, component localization, pose estimation, and failure handling carry influence across multiple high-weight customer requirements. The roof is used as a design-control device. For example, additional visual coverage can strengthen ES3 and ES4, but it increases calibration, synchronization, and data-management burden; similarly, recovery logic can improve ES5–ES6 while adding

controller integration work. These QFD trade-offs define the subsequent concept space. The platform and gripper must enable precise contact motion for ES1–ES2 and ES4; the tele-operator and data pipeline must produce usable demonstrations and observable scenes; and the policy must convert those observations into safe executable actions while leaving explicit hooks for failure monitoring and recovery.

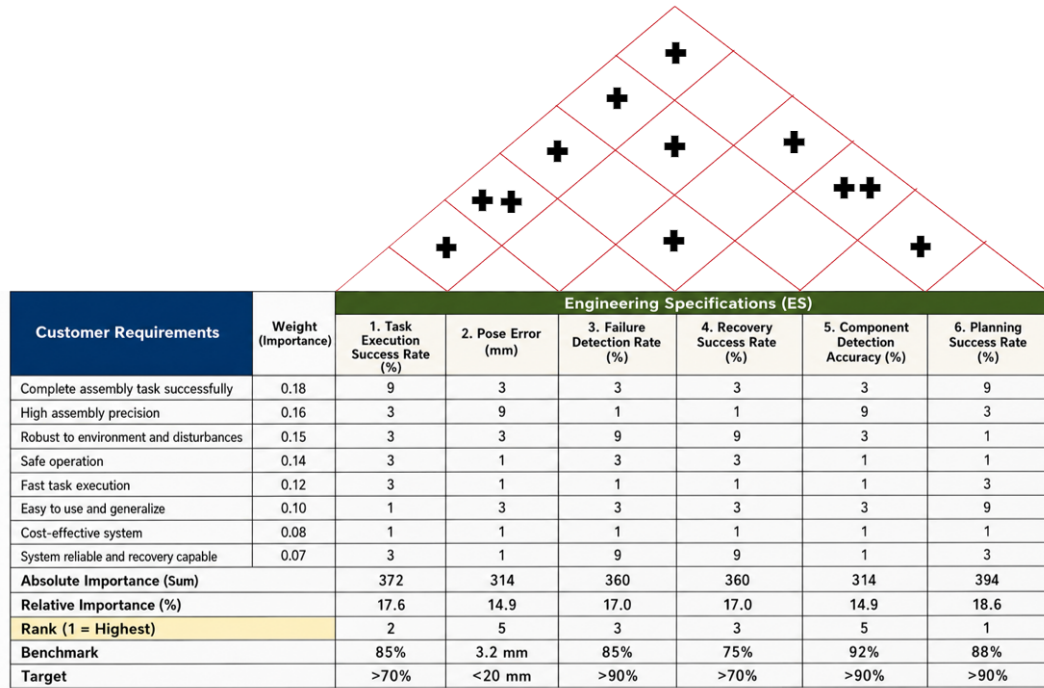


Illustration 2–1 QFD House of Quality for the VLA bimanual manipulation system.

Chapter 3 Concept generation and selection

3.1 Concept generation

3.1.1 Research landscape of imitation learning and vision-language-action models

Robot-learning methods for manipulation have developed along several related directions: demonstration-based visuomotor learning, temporal action-generation models, language-conditioned manipulation, and general-purpose vision-language-action (VLA) policies. These directions differ in the supervision they require, the observations they consume, the form in which they represent actions, and the extent to which one policy can cover multiple tasks. Reviewing them is important for concept generation because a policy cannot be selected independently of the demonstration interface, sensing arrangement, robot embodiment, and deployment constraints. In this project, the literature therefore defines the feasible policy and data-collection alternatives, while the QFD priorities and available dual-FR3 workcell determine which alternative is carried into implementation.

The field does not provide one universally superior controller. A method that performs well on broad tabletop benchmarks may still be unsuitable for contact-rich 3C assembly if its camera contract, action representation, inference rate, or robot interface cannot be reproduced on the target workcell. Conversely, a narrower imitation-learning method may be easier to deploy and validate but may require a separate checkpoint for each task. The relevant research landscape must consequently be interpreted as a set of engineering trade-offs rather than as a ranking based on headline benchmark values obtained under different embodiments and evaluation protocols.

3.1.1.1 Demonstration-based visuomotor learning

Imitation learning provides a direct route from human demonstrations to robot control. In its simplest form, behavior cloning learns a mapping from an observation and robot state to the action demonstrated by an operator. This formulation avoids the need to specify a reward function or to explore unsafe actions on the physical robot, which is attractive for industrial workcells. Its main limitation is distribution shift: small prediction errors can move the robot away from states represented in the demonstration data, after which later

predictions may become unreliable. Dataset coverage, action consistency, camera placement, episode screening, and the treatment of unsuccessful demonstrations are therefore part of the controller design rather than merely data-preparation details.

Robomimic showed that observation representation, training configuration, and demonstration quality materially affect offline robot-learning performance, making reproducible dataset construction a first-order design decision^[16]. MimicGen extends this direction by generating additional demonstrations from a smaller set of human examples, illustrating how task structure and data generation can be used to improve coverage without requiring every trajectory to be collected manually^[17]. These approaches motivate explicit episode boundaries, validation splits, and data-quality checks in a physical manipulation pipeline.

The demonstration interface also determines what action distribution can be learned. GELLO uses a low-cost joint-space leader whose kinematic correspondence with the follower robot supports intuitive trajectory collection^[18]. UMI instead emphasizes portable, in-the-wild demonstration capture and a reusable manipulation interface^[19]. These systems address different collection conditions, but both demonstrate that operator ergonomics, calibration, and mapping consistency affect the resulting dataset. For a fixed dual-FR3 work-cell, a joint-master interface offers a comparatively direct path to bilateral joint and gripper targets, whereas hand-held or immersive interfaces require an additional motion-retargeting layer. This distinction informed the teleoperation alternatives retained in the morphological chart.

3.1.1.2 Temporal imitation-learning and diffusion policies

Manipulation actions are temporally correlated, and predicting one independent command at a time can produce jitter or amplify short-term errors. Action Chunking with Transformers (ACT) addresses this issue by predicting a sequence of future actions and using a conditional variational formulation to represent variation among demonstrations^[20]. Action chunks can improve temporal coherence and reduce the effective prediction horizon for fine-grained manipulation. ACT is especially relevant to bimanual systems because the same output sequence can contain coordinated targets for both arms and grippers. Its practical advantages include a well-defined supervised-learning objective and an established route through LeRobot. Its principal limitation for the present application is that the conventional deployment route is task-specific: changing the requested operation normally requires selecting a

different checkpoint or adding a separate task-conditioning mechanism.

Diffusion Policy treats visuomotor control as conditional denoising over action trajectories^[21]. Rather than committing to one deterministic action in a potentially multimodal situation, the policy iteratively transforms noise into an action sequence conditioned on the current observation. Receding-horizon execution then allows the robot to execute part of the sequence and replan from updated observations. This structure is attractive for manipulation tasks in which several approach paths may be valid, but iterative denoising can increase inference cost and integration complexity. The final control quality also depends on whether the observation representation preserves the geometry needed near contact.

Subsequent work expands diffusion-based control toward richer spatial representations and larger-scale bimanual models. 3D Diffusion Policy incorporates compact three-dimensional observations to improve geometric reasoning and cross-task generalization^[22]. RDT-1B develops a diffusion foundation model specifically for bimanual manipulation and combines language, image, and proprioceptive inputs with a scalable action-generation architecture^[3]. These methods make diffusion policies strong candidates for contact-rich and bilateral tasks, but their reported results cannot be transferred directly to the present workcell without matching the camera observations, robot state, action convention, training data, and deployment interface. They were therefore treated as concept alternatives rather than assumed performance baselines.

3.1.1.3 Language-conditioned manipulation and visual representation

Before large VLA policies, several methods established how semantic instructions and spatial observations could be connected to robot actions. Transporter Networks represent pick-and-place actions through spatial feature transport, providing a strong inductive bias for rearrangement tasks^[5]. CLIPort combines a semantic pathway associated with language-conditioned object selection and a spatial pathway associated with precise placement^[6]. Per-Act maps language goals and RGB-D observations into a structured three-dimensional action representation, showing how language-conditioned manipulation can be formulated in a voxelized workspace^[7]. These methods are narrower than a general VLA policy, but they clarify an important design principle: semantic task understanding and local geometric precision are distinct requirements that may benefit from different observation representations.

Research on reusable visual features and multi-view geometry provides a complementary

perspective. R3M learns visual representations intended to transfer across downstream robot-manipulation tasks^[23], while RVT uses multi-view transformer representations for three-dimensional manipulation^[24]. Their relevance to the present project lies less in direct deployment than in the justification for combining global and wrist-local observations. A fixed overhead view preserves scene context and object relationships, whereas wrist views preserve detail near grasping and insertion. No representation can recover contact geometry that is consistently occluded or outside the field of view, so camera selection and synchronization remain coupled to policy selection.

3.1.1.4 Generalist vision-language-action models

VLA models extend language-conditioned manipulation by learning a common interface among visual observations, natural-language instructions, robot state, and continuous or tokenized actions. RT-2 demonstrated that knowledge represented in a vision-language model can be transferred to robot control by expressing actions in a token-compatible form and co-training on web and robot data^[1]. This line of work suggests that semantic knowledge acquired outside a single robot dataset can support task interpretation. However, semantic transfer does not by itself guarantee the millimetre-scale alignment, contact stability, or deterministic safety required by industrial insertion tasks.

OpenVLA provides an open-source VLA model and training route, making generalist robot policies more accessible for adaptation and comparative research^[2]. Its importance to concept generation is not simply model scale; it demonstrates a practical route for conditioning robot actions on images and language without designing a separate semantic module for each task. At the same time, adapting a general VLA model to a new embodiment still requires compatible action data, normalization, observation naming, and sufficient demonstrations of the target robot and task distribution.

The π family uses a flow-based action-generation approach for general robot control. The π_0 model combines vision-language representations with an action expert and predicts continuous robot actions through flow matching^[4]. The later $\pi_{0.5}$ model extends this direction toward open-world generalization and heterogeneous robot data while retaining language-conditioned action generation^[12]. The availability of OpenPI and a LeRobot $\pi_{0.5}$ integration further reduces the engineering distance between the published model and a reproducible project implementation^[13,15]. This software path is relevant because the final system requires

not only a capable model family but also a concrete method for mapping three named camera observations and a bilateral state vector to admissible dual-arm actions.

Generalist models nevertheless introduce limitations that are especially important in 3C assembly. Their broad semantic capability may exceed what is needed for three bounded tasks, while their computation and data requirements are greater than those of a small task-specific policy. Action chunks can also create an interval between observations during which local contact errors are not corrected. In addition, a probabilistic learned policy does not replace deterministic joint limits, workspace checks, emergency-stop handling, or robot-controller protections. These considerations mean that selecting a VLA model is a system decision involving data, sensing, temporal execution, and safety boundaries rather than a model-only upgrade.

3.1.1.5 Datasets, embodiment diversity, and evaluation

The development of general robot policies has been enabled by larger and more diverse datasets. BridgeData V2 provides a broad collection of manipulation trajectories and illustrates the value of task and scene diversity for reusable policies^[8]. Open X-Embodiment aggregates data across multiple robot embodiments and supports research on cross-embodiment transfer^[9]. DROID emphasizes large-scale, in-the-wild data collection across varied scenes, tasks, and operators^[10]. Together, these datasets shift the field from learning one behavior from one narrowly collected dataset toward pretraining representations and policies that can be adapted to new tasks.

Scale alone does not remove embodiment mismatch. Robots differ in kinematics, grippers, camera placement, control frequency, state definitions, and action conventions. A policy pretrained on heterogeneous data still requires a precise feature contract when deployed on a dual-FR3 system. Local demonstrations remain necessary to represent the target fixtures, object tolerances, bilateral role allocation, and absolute joint and gripper commands. Dataset balance is also important in multi-task training: without task-level sampling control, a task with longer episodes or more frames may dominate optimization even when all tasks should carry equal engineering importance.

Evaluation protocols must similarly separate offline agreement from physical task completion. LIBERO provides a benchmark for studying transfer and lifelong learning across language-conditioned manipulation tasks^[11], but benchmark success under a simulated or

standardized setting is not equivalent to end-to-end success on a physical industrial workcell. Offline losses are useful for checkpoint selection, while real-robot trials are required to evaluate grasping, alignment, insertion, collision-free execution, and task-specific terminal conditions. This distinction prevents literature benchmarks from being presented as measured evidence for the final hardware system.

3.1.1.6 Comparison and implications for concept generation

Table 3–1 summarizes the principal research directions that informed the policy and sensing concept space. The comparison is qualitative because the cited methods use different robots, datasets, action spaces, and evaluation protocols. It records the engineering role of each family rather than claiming a controlled benchmark comparison.

Table 3–1 Representative robot-learning directions relevant to concept generation

Direction	Representative methods	Primary inputs	Main strength	Limitation and project relevance
Spatial visuomotor learning	Transporter, CLIPort, PerAct ^[5-7]	Images, language, or structured 3D observations	Explicit semantic and spatial structure for object selection and placement	Provides design principles for perception and language grounding, but does not directly supply the implemented continuous bilateral action interface.
Sequence imitation learning	ACT ^[20]	Multi-view images and robot state	Temporally coherent action chunks and an established bimanual imitation-learning route	Practical DR3 baseline, but conventionally task-specific and dependent on demonstration coverage.
Diffusion control	Diffusion Policy, 3D Diffusion Policy, RDT-1B ^[3,21-22]	2D or 3D observations, state, and optionally language	Models multimodal action sequences and supports receding-horizon control	Strong policy alternative, with additional inference, representation, and workcell-integration requirements.
Generalist VLA	RT-2 and OpenVLA ^[1-2]	Images, language, and robot actions or state	Transfers semantic knowledge and supports a common language-conditioned policy interface	Broad capability does not guarantee local contact precision or compatibility with the target robot action schema.
Flow-based VLA	π_0 and $\pi_{0.5}$ ^[4,12]	Multi-view images, language, and proprioceptive state	Continuous action generation with a route to multi-task and multi-embodiment adaptation	Selected policy direction; requires local dual-FR3 data, strict feature mapping, and deterministic execution checks.
Large-scale robot data	BridgeData V2, Open X-Embodiment, DROID ^[8-10]	Heterogeneous demonstrations	Improves diversity and supports pretraining across tasks and robots	Does not replace task-specific data for the final fixtures, cameras, grippers, and bilateral action convention.

The literature leads to three implications for the present concept generation. First, the demonstration interface and observation pipeline must be selected together with the policy because they define the data distribution and feature contract available for learning. Second, ACT, diffusion policies, and VLA models represent credible but different trade-offs among deployment maturity, multimodal action generation, language conditioning, and computational burden. Third, an industrial proof-of-concept must evaluate the selected model on its own physical terminal conditions rather than infer performance from results reported on other embodiments.

These implications define the candidate space used below. ACT is retained as a mature action-chunking baseline, Diffusion Policy as an alternative trajectory-generation family, and $\pi_{0.5}$ as the language-conditioned route aligned with unified multi-task control. GELLO, multi-view RGB sensing, bilateral state and action recording, and deterministic robot-side checks complement these policy alternatives. The following functional decomposition converts this research-informed space and the QFD priorities into the five modules used for structured concept generation and selection.

3.1.2 Functional decomposition

The target assembly system was first decomposed into solution-neutral functions: establish the workspace state; create a stable robot–operator interface for demonstrations; record synchronized visual, proprioceptive, and action data; grasp and constrain the component during contact; map the current observation to coordinated motion; and detect or recover from an invalid execution state. The decomposition converts the QFD priorities into five design modules. The robot platform provides repeatable bimanual motion for ES1, ES2, and ES4; the teleoperation module enables usable demonstrations; the data-collection module supports ES3–ES4 and learning reproducibility; the gripper realizes component contact; and the policy maps observations and state to actions. Failure monitoring and recovery remain cross-cutting functions because they act across the collection and deployment loop.

3.1.3 Structured brainstorming

The team used structured brainstorming to generate alternatives for each module before choosing a configuration. Candidate ideas were retained when they addressed at least

one QFD-linked specification and did not violate laboratory safety, hardware access, or the project schedule. The session considered the available FR3 ecosystem alongside Flexiv and ALOHA-style platforms; direct joint-master, XR, and hand-held teleoperation; progressively richer observation schemas; compliant and rigid end-effectors; and policy families including Diffusion Policy, ACT, and the $\pi_{0.5}$ VLA model. This procedure deliberately separates the choice of a data-collection interface from the choice of a learning algorithm, since neither can be justified independently of the sensing and robot-control path.

The generated alternatives were also informed by established research directions rather than by ad hoc feature lists. Robomimic and MimicGen motivate treating demonstration quality and dataset construction as system-level design variables^[16-17]; RVT, 3D Diffusion Policy, and R3M respectively motivate the retained alternatives for multi-view geometry, 3D visuomotor control, and reusable robot visual representations^[22-24]. These references define the external concept space, while the selected configuration remains constrained by the laboratory hardware, the QFD priorities, and the project schedule.

3.1.4 Morphological chart and generated module concepts

Table 3–2 records the resulting concept space. A row represents one functional module, and a column entry represents one candidate realization. The bold entry in each row is the selected module concept; the table is not a claim that the unselected entries were all built or experimentally benchmarked.

Table 3–2 Morphological chart for the five functional modules

Concept	Candidate A	Candidate B	Candidate C
C1: Platform	Dual Franka FR3	Dual Flexiv platform	ALOHA-style bimanual platform
C2: Teleoperator	GELLO joint-master	PICO 4 Ultra XR	UMI hand-held
C3: Data collection	Two wrist + one top camera + robot state/action	One wrist + one top camera + robot state/action	Robot state/action only
C4: Gripper	Franka parallel gripper	3D-printed hard gripper	3D-printed soft gripper
C5: Policy	ACT in LeRobot	$\pi_{0.5}$ VLA in LeRobot	Diffusion policy

The morphological chart produces five leading module concepts, denoted C1–C5. C1 is a dual-FR3 platform; C2 is GELLO-based teleoperation; C3 is a synchronized three-camera, robot-state, and action pipeline; C4 is a 3D-printed soft gripper; and C5 is a unified $\pi_{0.5}$ VLA policy integrated through LeRobot. Together they form one integrated system concept, rather than five competing end-to-end robot systems. Their alternatives are retained because each exposes a real engineering trade-off: platform precision against integration cost, operator intuitiveness against mapping complexity, observation coverage against synchronization burden, compliance against geometric repeatability, and policy capability against data and deployment maturity. Detailed descriptions of the unselected alternatives are provided in the appendix.

3.2 Concept selection

3.2.1 Selection method and QFD-derived criteria

Selection was conducted in two stages. First, an absolute screen removed a candidate if it could not be made safe in the laboratory, could not interface with the selected robot and data path, or could not be obtained and integrated within the project schedule. Second, the surviving alternatives were scored on a five-point ordinal scale, where 1 denotes a weak fit and 5 denotes a strong fit. Ratings were assigned from the QFD relationship to ES1–ES6, laboratory availability and interface knowledge, the existing collection workflow, and the cited primary or official sources; no rating is presented as an unreported benchmark result. Consistent with the functional decomposition, the “top five concepts” are the five selected module concepts C1–C5. Their combination forms the single system concept carried into the final design. For candidate a in module m , the decision score is

$$S_{m,a} = \sum_{k=1}^4 w_k r_{m,a,k}, \quad (3-1)$$

where $r_{m,a,k}$ is the rating for criterion k and w_k is its QFD-derived weight. Scores are used only to compare alternatives within the same module; they are recorded design judgments, not measured task-performance results.

Table 3–3 QFD-derived criteria for module selection

No.	Criterion	Weight	QFD Link
SC1	Contribution to task performance and precision	0.30	ES1, ES2, ES3, ES4
SC2	Data, control, and software integration compatibility	0.25	ES1, ES2, ES3
SC3	Safety, robustness, and recovery support	0.20	ES5, ES6; CR3–CR4
SC4	Availability, manufacturability, and schedule feasibility	0.25	CR6–CR7 and project plan

3.2.2 Scoring of the five selected module concepts

Table 3–4 provides the detailed selection record for the five leading module concepts. The same criterion definitions and weights are applied in every module so that the scoring logic remains traceable to the QFD; however, the totals are interpreted only within a module because a robot platform, a gripper, and a policy make different contributions to the integrated system.

Table 3–4 Module-level concept selection matrix (1 = weak fit; 5 = strong fit)

Module	Candidate	SC1	SC2	SC3	SC4	Weighted score
C1 Platform	Dual Franka FR3	5	5	4	3	4.30
	Dual Flexiv	4	3	4	2	3.30
	ALOHA-style bimanual	3	4	3	5	3.85
C2 Teleoperator	GELLO joint-master	4	5	4	5	4.50
	PICO 4 Ultra XR	3	3	3	4	3.25
	UMI hand-held	3	3	3	4	3.25
C3 Data collection	Two wrist + top camera + state/action	5	5	4	3	4.35
	One wrist + top camera + state/action	4	4	4	5	4.20
	Robot state/action only	2	3	3	5	3.25
C4 Gripper	3D-printed soft gripper	4	4	4	5	4.20
	Franka parallel gripper	3	5	4	3	3.70

Module	Candidate	SC1	SC2	SC3	SC4	Weighted score
	3D-printed hard gripper	4	5	3	4	4.05
C5 Policy	$\pi_{0.5}$ VLA in LeRobot	5	5	4	4	4.55
	ACT in LeRobot	4	5	4	5	4.50
	Diffusion Policy	4	3	4	4	3.75

3.2.3 Comparative evaluation and selected concept

C1: Dual Franka FR3 platform. The dual Franka FR3 platform was selected as the mechanical foundation of the final system because it provides repeatable 7-DoF control, a direct connection to the available laboratory control ecosystem, and a credible path toward coordinated bimanual manipulation. Its principal disadvantage is the substantial integration effort required for workspace coordination, collision avoidance, commissioning, and bilateral control. Nevertheless, these costs are justified by the project objective: lower-cost alternatives may reduce initial hardware burden, but they would introduce a different control ecosystem or provide a weaker fit to the required assembly precision.

C2: GELLO teleoperation. GELLO was selected and implemented as the demonstration-collection interface because it offers a direct and intuitive connection between operator motion and the robot-state/action records required by the learning pipeline^[18]. Despite the need for reliable leader–follower mapping and calibration, GELLO remains preferable to immersive or hand-held alternatives for the present system because it provides the most direct route from teleoperated demonstrations to the joint-space data representation used by the FR3 and LeRobot workflow.

C3: Three-camera data pipeline. The selected sensing configuration uses one Intel RealSense D435 overhead camera and two Intel RealSense D405 wrist-mounted cameras, together with synchronized robot state, gripper state, and action records. The D435 provides a stable global view of the shared workspace, while the two D405 cameras preserve local visual information near the end-effectors and contact regions. This complementary arrangement reduces visual blind spots and directly supports the component-detection and pose-estimation requirements in ES3–ES4, at the cost of increased synchronization and data-management effort.

C4: 3D-printed soft gripper. The selected end-effector is a 3D-printed soft gripper derived from the open-source gripper design in the referenced UMI material; the project adopts only the gripper-finger component rather than reproducing the complete UMI hardware system^[19]. Compliant contact and rapid geometry iteration are valuable for handling assembly components with small geometric variation, although deformation may reduce pose repeatability compared with a rigid locating gripper.

C5: Unified $\pi_{0.5}$ VLA policy in LeRobot. ACT was initially selected and deployed as the task-specific policy baseline because it was already integrated with the LeRobot workflow and provided a traceable route from synchronized visual observations and robot-state inputs to action-chunk execution^[14,20]. As the project progressed beyond the DR3 baseline, the policy selection was updated to the LeRobot implementation of $\pi_{0.5}$ ^[12-13,15]. The final policy receives three synchronized RGB observations, the bilateral robot state, and a language instruction, and predicts bilateral robot and gripper actions. One jointly trained checkpoint is used for the ring, plug-insertion, and memory-module tasks; task identity is provided through the corresponding language instruction rather than by loading separate task-specific checkpoints. Unified language-conditioned multi-task bimanual control was implemented and validated in 75 physical trials, as reported in Chapter 5. ACT and Diffusion Policy are therefore retained as earlier baselines rather than presented as the final deployed policy.

The selected architecture is the combination of the highest-scoring surviving candidate in each module. C1 and C4 establish the physical capability for controlled contact; C2 and C3 make demonstrations and visual evidence available to the learner; and C5 provides the deployable policy path. This combined decision covers the full ES1–ES6 chain while retaining a feasible implementation route for subsequent final design, manufacturing, and validation.

Chapter 4 Final design and implementation

4.1 Final design and engineering changes

4.1.1 Design evolution from the DR3 baseline

The final implementation retains the modular physical architecture selected in Design Review 3 (DR3), but the learning system and demonstrated scope have changed substantially. DR3 proposed a dual-Franka Research 3 (FR3) workcell, GELLO demonstration collection, two wrist cameras and one overhead camera, custom compliant fingers, a LeRobot dataset, and an imitation-learning policy. At that stage, Action Chunking with Transformers (ACT) and task-specific checkpoints were the deployable baseline. Language-conditioned bimanual control was a future objective.

The completed system replaces the task-specific ACT controller with the LeRobot implementation of $\pi_{0.5}$. It uses two FR3 arms, two grippers, three RGB views, a 16-dimensional bilateral state and action interface, and one jointly trained multi-task checkpoint. The three tasks are selected by natural-language instructions rather than by manually loading three independent policy files. The final physical evaluation used the 65,000-step checkpoint selected by the task-balanced offline validation rule. ACT therefore remains part of the design history and concept-selection evidence, but it is not presented as the final controller.

The mechanical elements selected in DR3 remain directly relevant. The two FR3 arms provide a common kinematic and controller interface, and the two GELLO leaders provide matched bilateral demonstration inputs. The wrist and overhead cameras provide complementary local and global views. The 3D-printed compliant fingers provide a task-compatible contact interface. The later policy update changes how these modules are connected and trained, but does not remove the need for their original engineering analysis.

4.1.2 Engineering Change Notice from DR3 to the final prototype

Table 4–1 records the material changes between the DR3 baseline and the implemented prototype. The purpose of this Engineering Change Notice (ECN) is not to restate the project history. It identifies which interfaces changed, why the change was accepted, which evidence verifies the final state, and which limitation remains after implementation.

Table 4-1 Engineering Change Notice from the DR3 baseline to the final prototype

ECN	DR3 baseline	Final implemented state	Reason and affected interface	Verification and residual risk
01	Task-specific ACT checkpoints selected by the operator.	One language-conditioned LeRobot $\pi_{0.5}$ checkpoint is shared by all three tasks.	The change removes manual policy-file switching and makes the instruction, three images, and bilateral state one common policy input.	The 65,000-step checkpoint completed all three physical tasks. No matched ACT trial set was run, so comparative superiority is not claimed.
02	Single-arm and staged validation evidence was used while the dual-arm route remained the target architecture.	Two FR3 arms and two grippers are represented by one 16-dimensional bilateral state and one 16-dimensional absolute action.	The data, model adapter, and robot adapter now use the same $7 + 1 + 7 + 1$ ordering.	Dataset metadata, checkpoint configuration, dry runs, and physical trials verify the interface. Exact base poses and swept-volume geometry remain documentation gaps.
03	Earlier policy work used task-specific datasets and did not establish a common multi-task sampler.	<code>dual_ring</code> , <code>dual_socket</code> , and <code>dual_ram</code> are trained jointly with equal task probability.	Equal task probability prevents the ring dataset, which has the most frames, from dominating task selection during training.	The dataset manifest and sampler configuration verify the three-task scope. The compatible <code>dual_socket2</code> directory was not included.
04	Manual checkpoint identity was coupled to the selected task.	Checkpoint selection uses the unweighted macro flow-matching loss of the three tasks; step 65,000 is selected.	A task-balanced offline rule is used before physical testing, avoiding selection from the final 75 task outcomes.	Step 65,000 has macro loss 0.036552. Offline loss remains a model-selection quantity, not a physical success metric.

ECN	DR3 baseline	Final implemented state	Reason and affected interface	Verification and residual risk
05	ACT predicted a 32-step chunk and the earlier executor operated under a different policy-specific temporal interface.	$\pi_{0.5}$ predicts 50 actions and the executor consumes the first 10 at the configured 5 Hz cadence before replanning.	The longer prediction horizon supplies temporal context while the shorter executed segment limits open-loop motion.	Shape and queue behavior are verified in dry runs and deployment. End-to-end query latency was not measured.
06	DR3 listed planned ES1–ES6 validation activities.	The final report contains completed ES1–ES6 results and 75 counted end-to-end task trials.	The validation record now separates planning, task execution, detection, pose error, failure detection, and recovery rather than treating one aggregate rate as complete validation.	Chapter 5 reports denominators and pass/fail judgments. The ES4 raw error series is unavailable and is not reconstructed.
07	The current controller and source state were described primarily by a repository branch and intended scripts.	The report records the actual local training files, environment versions, file hashes, checkpoint assets, and uncommitted-source risk.	Software, model weights, normalization assets, and run configuration are treated as virtual implementation materials.	The final checkpoint is portable, but the Git commit alone cannot reproduce the training run because relevant files were untracked or locally modified.
08	The proof-of-concept budget emphasized zero incremental cash expenditure.	The final budget separates zero additional cash cost from in-kind robots, cameras, compute, storage, fabrication, software, and labour.	The distinction prevents a laboratory system from being represented as having zero economic value.	The resource categories are recorded; replacement values, print time, energy, and labour were not supplied.

The ECN shows that the central architectural change is concentrated in the learning and validation layers. The two-arm mechanical platform, sensing layout, demonstration interface, compliant fingers, and deterministic robot-side protection remain continuous with DR3. This

continuity makes the physical workcell a controlled basis for evaluating the new policy route, but it does not create a controlled ACT-versus- $\pi_{0.5}$ comparison because the earlier and final controllers were not tested under one matched protocol.

4.1.3 Overall system architecture

The final system is organized as four connected layers:

1. the physical workcell, consisting of two FR3 arms, two custom grippers, task fixtures, and three cameras;
2. the demonstration layer, consisting of the two GELLO leader devices, the ROS 2 bridge, and the episode recorder;
3. the learning layer, consisting of LeRobot Dataset v3.0, the $\pi_{0.5}$ training configuration, and task-balanced validation; and
4. the deployment layer, consisting of language-conditioned policy inference, action-chunk handling, the bilateral robot adapter, and deterministic safety checks.

During demonstration collection, the operator moves the two GELLO leaders. The follower-side system commands the two FR3 arms. It records three RGB images, the measured bilateral state, the bilateral arm targets, the gripper commands, the episode index, and the task instruction. Accepted episodes are stored in LeRobot Dataset v3.0 format. During training, the three task datasets are sampled with equal task probability and used to update one $\pi_{0.5}$ model. During deployment, one of the three language instructions and the current three-view observation are supplied to the same checkpoint. The policy returns an action chunk, and the robot-side adapter sends the admissible absolute targets to the two FR3 controllers.

Figure 4–1 summarizes the final $\pi_{0.5}$ training and action-generation architecture. Three synchronized camera views are converted into multi-view visual tokens, while the task instruction and 16-dimensional bilateral proprioceptive state provide language and robot-state conditioning. The conditioned representation is processed by the adapted $\pi_{0.5}$ model and its robot action expert to generate a continuous 16-dimensional bimanual action chunk containing absolute joint targets and gripper widths for the left and right arms. The figure therefore represents the implemented language-conditioned multi-task policy interface rather than the earlier DR3 ACT baseline.

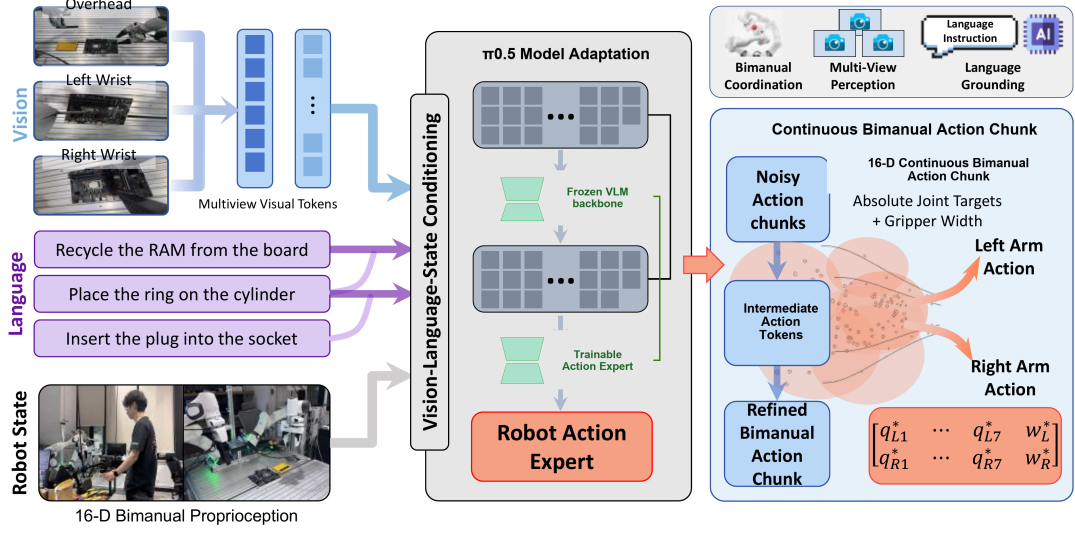


Illustration 4-1 Final $\pi_{0.5}$ system overview for language-conditioned bimanual manipulation. Three camera views, a task instruction, and the 16-dimensional bilateral robot state condition the frozen vision-language backbone and the adapted robot-action path, which produces continuous absolute joint and gripper action chunks for both FR3 arms.

4.1.4 VLA input-output formulation and notation

The final controller can be expressed as a conditional visual-language-action policy. At decision time t , the visual observation is the ordered set

$$\mathcal{I}_t = \left\{ \mathbf{I}_t^{\text{overhead}}, \mathbf{I}_t^{\text{left}}, \mathbf{I}_t^{\text{right}} \right\}, \quad (4-1)$$

the language instruction is ℓ , and the bilateral proprioceptive state is $\mathbf{s}_t \in \mathbb{R}^{16}$. The learned policy maps these conditions to a 50-step bilateral action chunk:

$$\hat{\mathbf{A}}_{t:t+H-1} = \pi_{\theta}(\mathcal{I}_t, \ell, \mathbf{s}_t), \quad \hat{\mathbf{A}}_{t:t+H-1} \in \mathbb{R}^{H \times 16}, \quad H = 50. \quad (4-2)$$

Each row contains the absolute targets for seven left-arm joints, the left gripper, seven right-arm joints, and the right gripper. Equation 4-2 is the system-level VLA contract; it does not imply that the model directly controls joint torque or bypasses the FR3 low-level controllers.

The executor applies only the first $h = 10$ rows:

$$\hat{\mathbf{a}}_{t+j} = \left[\hat{\mathbf{A}}_{t:t+H-1} \right]_{j,:}, \quad j = 0, \dots, h-1, \quad h = 10, \quad (4-3)$$

then acquires a new $\mathcal{I}_{t+h}$ and $\mathbf{s}_{t+h}$ and queries the policy again. The predicted horizon is therefore longer than the executed segment. This receding-horizon use of action chunks limits the amount of motion committed before visual and proprioceptive feedback is refreshed.

Table 4–2 Principal symbols used in the $\pi_{0.5}$ design and validation equations

Symbol	Dimension or domain	Meaning
t	decision index or sampled flow time	Context-dependent time variable; the flow-time use is stated explicitly in the training subsection.
$\mathcal{I}_t$	three RGB images	Ordered overhead, left-wrist, and right-wrist visual observation.
ℓ	token sequence	Natural-language task instruction supplied to the shared checkpoint.
$\mathbf{s}_t$	$\mathbb{R}^{16}$	Bilateral measured joint and gripper state.
$\mathbf{a}_t$	$\mathbb{R}^{16}$	One bilateral absolute joint/gripper target.
$\hat{\mathbf{A}}_{t:t+H-1}$	$\mathbb{R}^{H \times 16}$	Predicted bilateral action chunk.
H	50 action steps	Model prediction horizon.
h	10 action steps	Number of actions consumed before re-observation.
θ	model parameters	Parameters of the conditional $\pi_{0.5}$ policy; only the action path and projections are trained in the final run.
ϵ	$\mathbb{R}^{50 \times 16}$	Gaussian noise used by the flow-matching objective.
$\mathbf{x}_t$	$\mathbb{R}^{50 \times 16}$	Noisy interpolated action state at sampled flow time t .
$\mathbf{u}_t$	$\mathbb{R}^{50 \times 16}$	Target flow velocity $\epsilon - \mathbf{a}$.
$\mathbf{v}_\theta$	$\mathbb{R}^{50 \times 16}$	Action-expert velocity prediction.
$\mathcal{L}_{\text{FM}}$	scalar	Normalized flow-matching training loss.
$\mathcal{L}_{\text{macro}}$	scalar	Unweighted mean of ring, socket, and RAM validation losses.
$\hat{p}$	$[0, 1]$	Observed success proportion x/n in Chapter 5.

4.1.5 Selected module concepts

4.1.5.1 Dual-FR3 bimanual platform

Two FR3 arms are used as the follower robots. The use of the same robot model on both sides reduces interface asymmetry: each side contributes seven joint positions, receives seven absolute joint targets, and uses the same low-level controller family. The shared kinematic structure also makes bilateral indexing, joint-limit validation, and data normalization easier to audit. Bimanual operation is required because each task includes a preparation or

stabilization action followed by a more precise manipulation action.

4.1.5.2 GELLO demonstration interface

The demonstration interface uses one GELLO leader for each follower arm. GELLO provides a physical leader whose joint arrangement is mapped to the follower robot, giving the operator direct control of coordinated arm motion^[18]. The selected project configuration provides seven arm channels and one gripper channel per leader. Its low-cost modular construction also allows damaged or geometrically unsuitable leader components to be reprinted without changing the follower robot.

4.1.5.3 Three-camera visual observation

The final observation contains one fixed overhead view and two wrist views. The overhead Intel RealSense D435i observes the shared workspace, fixture positions, and large object motion. The two Intel RealSense D405 cameras move with the left and right wrists and preserve local visual information near grasp, contact, insertion, extraction, and placement regions. This arrangement avoids relying on a single global view that may be occluded by either arm, while avoiding a wrist-only configuration that loses global task context.

4.1.5.4 Custom compliant grippers

Each arm uses the custom gripper developed during DR3. An assembly contains two identical screw-mounted fingers printed in TPU 64D. Each finger has an overall bounding-box size of

$$123.36 \times 25.80 \times 40.00 \text{ mm.}$$

The tapered outline and repeated ribs permit passive deformation around the handled ring, the insulated plug body, the motherboard edge, and the memory module. The reinforced mounting end keeps the screw interface stiffer than the contact region. Compliance is used to increase geometric tolerance; it is not treated as a substitute for camera coverage, accurate demonstration, or robot-side contact limits.

4.1.5.5 LeRobot $\pi_{0.5}$ policy

The final policy is the LeRobot implementation of $\pi_{0.5}$, adapted from the open-source OpenPI implementation^[12-15]. It receives three RGB images, the current bilateral state, and a language instruction. It predicts a chunk of bilateral actions in the same absolute joint-and-

gripper representation used by the recorder and executor. The same model parameters are used for all three tasks.

4.1.6 Operational workflow

4.1.6.1 Workcell initialization

Before an episode or an autonomous trial begins, the two arms are returned to the documented initial joint region, the grippers are inspected, and the task objects are reset. Camera mounts, image identities, network connections, controller status, workspace boundaries, and emergency-stop access are checked. The socket fixture is kept electrically unpowered. A run is not started if a camera is missing, a robot state is stale, or the scene does not match the selected instruction.

4.1.6.2 Demonstration collection

The operator performs the selected task using the two GELLO leaders. The ROS 2 bridge assembles packets containing the three images, the measured robot and gripper state, and the commanded arm and gripper values. The recorder samples these packets at 5 Hz and stores a complete episode. The operator then reviews task completion and resets the workcell before the next episode. Episode boundaries are preserved so that complete trajectories, rather than individual frames, can be assigned to training or validation.

4.1.6.3 Dataset screening and training

The recorded episode is checked for the presence and consistent length of all three image streams, the 16-dimensional state, the 16-dimensional action, the task identity, and the episode metadata. Accepted episodes are added to one of the three task datasets. Training then samples the three tasks with equal probability, applies the defined image and numeric preprocessing, and fine-tunes one $\pi_{0.5}$ checkpoint for 100,000 optimizer steps.

4.1.6.4 Closed-loop deployment

At deployment, the operator selects the natural-language instruction that matches the prepared scene. The model receives the instruction, three current images, and the current bilateral state. It predicts 50 actions, of which the first 10 are consumed before a new observation is evaluated. The executor checks the returned targets and sends them to the two arms at the

dataset-aligned control rate. A task terminates on completion, timeout, invalid observation, invalid action, protective stop, emergency stop, or manual intervention.

4.2 Engineering design analysis

4.2.1 Relevant engineering fundamentals

The implemented system combines serial-manipulator kinematics, shared-workspace bi-manual coordination, compliant contact, multi-view machine perception, imitation learning, packet-level data synchronization, and receding-horizon control. The learned policy produces high-level position targets. It does not replace the FR3 low-level servo controller, joint-limit enforcement, protective-stop behavior, or the external emergency-stop procedure.

For one seven-degree-of-freedom arm with joint vector

$$\mathbf{q}_t = [q_{1,t}, \dots, q_{7,t}]^\top,$$

the end-effector pose is

$${}^B\mathbf{T}_{E,t} = f(\mathbf{q}_t), \quad (4-4)$$

where B is the robot-base frame, E is the end-effector frame, and $f(\cdot)$ is the forward-kinematics map. In the dual-arm system, the corresponding transforms are

$${}^{B_L}\mathbf{T}_{E_L,t} = f_L(\mathbf{q}_t^L), \quad {}^{B_R}\mathbf{T}_{E_R,t} = f_R(\mathbf{q}_t^R). \quad (4-5)$$

If both base poses are expressed in a common workcell frame W , the relative end-effector transform can be written as

$${}^{E_L}\mathbf{T}_{E_R,t} = ({}^W\mathbf{T}_{B_L} {}^{B_L}\mathbf{T}_{E_L,t})^{-1} {}^W\mathbf{T}_{B_R} {}^{B_R}\mathbf{T}_{E_R,t}. \quad (4-6)$$

Equation 4–6 explains why the two base installations and task fixtures must remain fixed: the learned policy observes images and joint states rather than an explicitly recalibrated relative pose at every episode.

4.2.2 Design assumptions and constraints

The system operates in a structured indoor workcell. Lighting, camera mounts, robot bases, and task fixtures remain sufficiently repeatable for the learned visual policy. Each

trial begins within the demonstrated workspace. The can, handled ring, socket box, plug, motherboard, memory module, and yellow tray are treated as task-specific laboratory objects rather than arbitrary unseen industrial components.

The current model uses RGB images and bilateral proprioception. Although the selected RealSense devices can provide depth, and the D435i includes inertial sensing, depth and inertial measurements are not included in the implemented $\pi_{0.5}$ feature contract. The chapter therefore uses manufacturer depth ranges and fields of view to justify camera placement, but it does not claim that the policy performs explicit metric depth reconstruction or six-degree-of-freedom pose estimation.

The recorder relies on software packets assembled by an upstream ROS 2 bridge. It does not use a hardware trigger or exposure-level synchronization. It also does not perform device-time interpolation or ROS approximate-time alignment. The valid engineering claim is packet-level synchronization, not exact simultaneity of the three camera exposures.

The design also assumes that deterministic safeguards remain outside the policy. The learned action is accepted only after dimensional, finite-value, joint, gripper, workspace, freshness, and communication checks. The socket task uses an electrically unpowered fixture. The system is a supervised research workcell and is not represented as a certified production cell.

4.2.3 FR3 parameters and bimanual kinematic suitability

Each FR3 provides seven degrees of freedom, a nominal payload of 3 kg, a maximum reach of 855 mm, and link-side torque sensing in all seven axes^[25-27]. These values are suitable for the project because the manipulated objects are light, the task is performed on a tabletop, and the seventh joint gives redundancy for changing wrist orientation while avoiding fixtures and the other arm.

The complete joint ranges, mounting specification, environmental limits, controller interfaces, and dimension drawings are reference data rather than design arguments. They are collected in Appendix 2, Table 2-1, and Figures 2-1-2-2. The practical task workspace is deliberately smaller than those hardware bounds so that action chunks do not intentionally approach joint limits.

Figure 4-2 is retained in the main analysis because it directly supports placement of the

Table 4–3 FR3 parameters relevant to the selected bimanual workcell

Parameter	Value	Design relevance
Degrees of freedom	7 per arm	Provides redundant approach configurations and wrist orientation control.
Nominal payload	3 kg	Exceeds the mass required for the project objects and custom fingers.
Maximum reach	855 mm	Supports a shared central tabletop region when the two bases are arranged on opposite sides.
Joint sensing	Link-side torque sensing in all seven axes	Supports compliant low-level control, contact monitoring, and protective behavior.

shared task region. The 855 mm maximum reach is not treated as an invitation to place objects at the boundary: motion near maximum extension reduces available orientation freedom and can bring the arm closer to singular configurations.

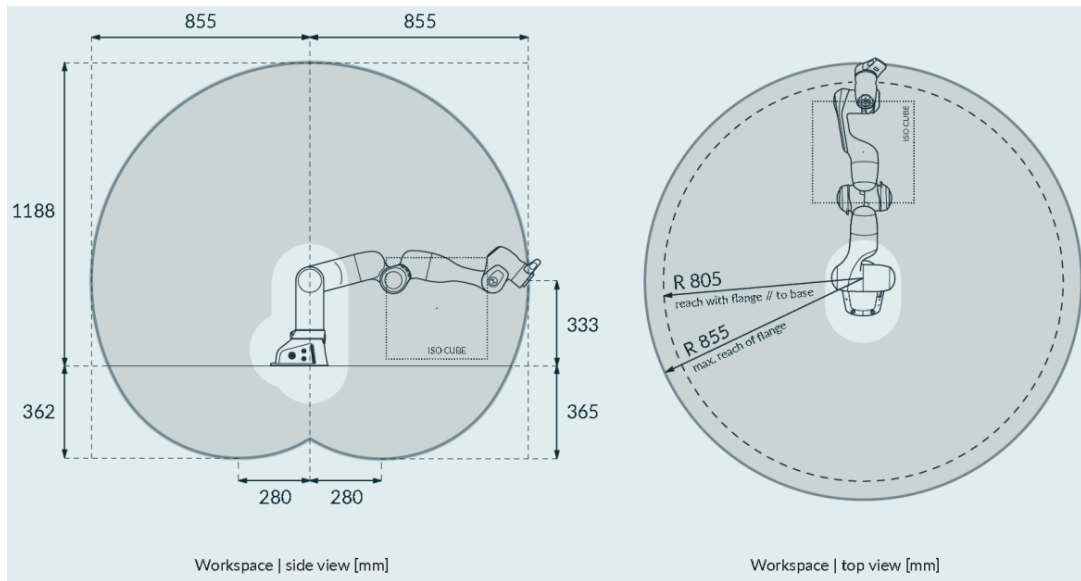


Illustration 4–2 Side and top views of the FR3 workspace envelope. The shared task region is selected inside the overlap of the two practical arm workspaces rather than at the 855 mm reach boundary.

4.2.4 Bimanual state and action representation

The actual state and action are both stored as 16-dimensional `float32` arrays. The state is

$$\mathbf{s}_t = [q_{1,t}^L, \dots, q_{7,t}^L, g_t^L, q_{1,t}^R, \dots, q_{7,t}^R, g_t^R]^\top \in \mathbb{R}^{16}, \quad (4-7)$$

where g_t^L and g_t^R are measured gripper opening widths in metres. The action uses the same ordering:

$$\mathbf{a}_t = [q_{1,t+1}^{L,\text{target}}, \dots, q_{7,t+1}^{L,\text{target}}, g_t^{L,\text{target}}, q_{1,t+1}^{R,\text{target}}, \dots, q_{7,t+1}^{R,\text{target}}, g_t^{R,\text{target}}]^\top. \quad (4-8)$$

Indices 0–6 correspond to the left arm, index 7 to the left gripper, indices 8–14 to the right arm, and index 15 to the right gripper. Arm actions are absolute next-step joint targets, not joint increments. Gripper actions are absolute `open_width_m` commands. The implemented setting is therefore `use_relative_actions=false` in both training and deployment.

The $\pi_{0.5}$ implementation pads state and action features to an internal maximum of 32 dimensions. Only entries 0–15 contain project data. Entries 16–31 are padding and are never interpreted as additional robot commands. The robot adapter explicitly slices and validates the first 16 entries before decomposing them into left-arm, left-gripper, right-arm, and right-gripper targets.

4.2.5 Shared workspace and role allocation

The two arms operate in a shared central region, so task role allocation is selected to reduce unnecessary crossing. In Task 1, the left arm pushes the can toward the centre while the right arm grasps and places the handled ring. In Task 2, the right arm pushes the socket box while the left arm grasps and inserts the plug. In Task 3, the left arm grips and pulls the motherboard while the right arm removes the memory module and places it in the yellow tray.

These assignments separate the large preparation motion from the subsequent precision motion. They also preserve the view of the precision-side wrist camera: the wrist that performs the ring placement, plug insertion, or RAM extraction approaches the relevant contact region directly. Role allocation is encoded by the demonstrations and the fixed action indexing; it is not dynamically reassigned by a separate planner.

The workcell dimensions and exact base-to-fixture transforms were not supplied as final measured values. Consequently, this report does not invent a dimensioned collision-free volume. The implemented engineering rule is that fixtures are placed in an experimentally verified overlap region, nominal trajectories stay away from the manufacturer reach boundary, and the robot-side workspace limits are tighter than the theoretical workspace shown in Figure 4–2. A production release would require the final base poses, fixture coordinates, swept-volume analysis, and inter-arm collision geometry to be documented numerically.

4.2.6 Contact, clearance, and insertion tolerance analysis

4.2.6.1 Ring-over-can placement

The ring task contains three mechanically distinct phases: a planar push of the can, a grasp of the ring handle, and a vertical placement of the ring over the can. Successful placement requires the ring inner opening to remain clear of the can outer profile throughout descent. If D_r is the effective ring inner diameter and D_c is the effective can outer diameter, the ideal radial clearance is

$$c_r = \frac{D_r - D_c}{2}. \quad (4-9)$$

The allowable lateral placement error must be smaller than c_r , with an additional margin for ring tilt, finger deformation, and uncertainty in the apparent object boundary. The actual D_r and D_c values were not provided, so Equation 4–9 defines the required measurement rather than claiming a numerical tolerance.

The handle allows the right gripper to avoid pinching the ring circumference, but it introduces an offset between the grasp point and the ring centre. This offset creates a moment that can tilt the ring during transport. A near-vertical final approach and release after the ring is supported by the target reduce the sensitivity to that moment.

4.2.6.2 Plug insertion

Plug insertion is a constrained contact task. Lateral and angular misalignment can produce edge contact, jamming, incomplete seating, socket-box displacement, or excessive force. For an insertion depth L_i and effective lateral clearance c_i , a first-order small-clearance angular bound is

$$\theta_{\max} \approx \tan^{-1} \left(\frac{c_i}{L_i} \right). \quad (4-10)$$

The relation shows that angular tolerance decreases as insertion depth increases. The socket box is pushed into a repeatable central region before insertion so that the left arm can approach with sufficient remaining wrist orientation range. The right-arm push must end before the left-arm precision approach enters the same local region.

The DR3 socket mock-up was electrically unpowered and had a recorded overall size of $80 \times 80 \times 30$ mm. The current task continues to use an unpowered socket fixture. The exact dimensions of the final socket box, pin clearance, insertion depth, and fixture compliance should be remeasured if the physical object differs from the DR3 mock-up. Until those values are recorded, success is defined functionally: the correct plug enters the correct socket, remains seated without manual assistance, and produces no visible damage.

4.2.6.3 Memory-module extraction

The motherboard task requires fixture stabilization, connector release, module extraction, and safe placement. The left gripper pulls the motherboard closer and holds or stabilizes it. The right gripper must approach the memory module without contacting nearby components, apply a controlled extraction motion consistent with the connector geometry, and transport the released module to the yellow tray.

The mechanical tolerance is governed by the free space around the memory module, the width and compliance of the custom fingers, and the connector's extraction direction. Excess lateral motion can bend the module or load adjacent board components. Because connector retention force, neighbouring-component clearance, and module dimensions were not supplied, the report does not infer extraction force or a numerical clearance. These quantities remain required measurements for a complete contact-mechanics verification.

4.2.7 Custom gripper geometry and mechanical analysis

Each gripper assembly uses two identical TPU 64D fingers, and the dual-arm workcell therefore uses four fingers. For one finger,

$$L = 123.36 \text{ mm}, \quad T = 25.80 \text{ mm}, \quad H = 40.00 \text{ mm}.$$

The principal geometric ratios are

$$\frac{L}{T} = \frac{123.36}{25.80} \approx 4.78, \quad \frac{L}{H} = \frac{123.36}{40.00} \approx 3.08. \quad (4-11)$$

These ratios describe an elongated finger rather than a short rigid jaw. The repeated ribs reduce local bending stiffness along the contact region, the tapered outline improves access, and the thicker mounting end distributes screw load.

The same compliance has a trade-off. It accommodates object-shape and pose variation and can reduce concentrated contact pressure, but deformation can reduce grasp repeatability and make the effective fingertip pose load dependent. The task design therefore uses compliance for tolerance while retaining fixed camera identities, repeatable starts, and arm-side position control. No finite-element stress, fatigue-life, or calibrated force-deflection result is claimed because print orientation, infill, wall count, actual TPU properties, contact loads, and cycle count were not supplied. The full orthographic drawing and fabrication-reference dimensions are provided in Appendix 2, Figure 2–4.

4.2.8 Perception working distance, field of view, and camera selection

The overhead camera is an Intel RealSense D435i. Its specified ideal depth range is approximately 0.3–3 m and its depth field of view is approximately $87^\circ \times 58^\circ$ ^[28]. This makes it suitable for observing the complete tabletop from a fixed elevated position. The overhead view supplies global information when either wrist view is occluded or too close to show the relationship between the object, fixture, and destination.

Each wrist camera is an Intel RealSense D405. Its specified ideal depth range is approximately 7–50 cm, with an approximately $87^\circ \times 58^\circ$ depth field of view^[29]. This range is better matched to a camera mounted near the end effector. The wrist views preserve local detail during ring descent, plug alignment, and memory-module extraction.

The hardware can operate at higher image and depth rates than the learning pipeline, but the project records RGB at 424×240 and 5 Hz. The selected policy does not consume depth or D435i inertial data. Camera range and field of view therefore justify coverage and mounting, while the learned input remains three RGB images.

4.2.9 Image format and preprocessing

All three raw streams have width 424 and height 240. The LeRobot dataset stores each decoded RGB image in channel-first form:

$$[C, H, W] = [3, 240, 424].$$

Table 4–4 Camera configuration and selection basis

Camera	Quantity	Nominal range	ideal	Depth FOV	Project function
RealSense D435i	1	0.3–3 m		$87^\circ \times 58^\circ$	Fixed overhead context for the shared workcell, fixtures, destinations, and large object displacement.
RealSense D405	2	7–50 cm		$87^\circ \times 58^\circ$	Left- and right-wrist close-range views for grasp, contact, insertion, extraction, and placement.

Before an image is passed to $\pi_{0.5}$, it is converted to a $3 \times 224 \times 224$ tensor. The conversion preserves aspect ratio. The 424×240 image is scaled to approximately 224×126 , and the remaining square area is filled with approximately 49 black pixels above and below the resized image. The image is not stretched to 224×224 .

Pixel values are represented in $[0, 1]$ after decoding and are normalized to $[-1, 1]$ before the model. This process is applied independently but identically to the overhead, left-wrist, and right-wrist images. Camera names and ordering are part of the feature contract. A swapped wrist-camera mapping can produce a dimensionally valid but semantically incorrect policy input, so identity is checked before recording and deployment.

4.2.10 GELLO channel architecture and leader–follower mapping

The bilateral collection system uses two GELLO–Franka leaders. Each leader provides seven arm channels and one gripper-related channel, matching the seven joint targets and one gripper command required by its FR3 follower. This channel correspondence is the design-relevant property; the detailed actuator, controller-board, power-supply, structural-kit, and dual-system quantities are listed in Appendix 2, Table 2–2. The leader drawing and actuator reference data are also moved there.

Leader motion must be mapped to follower joint order, sign convention, zero offset, and

permitted range. An incorrect offset can generate a valid numeric command that is unsafe for the follower. The pre-collection procedure therefore verifies the leader zero, moves each channel through a small range, confirms the corresponding follower direction, and checks the gripper-width mapping before a task episode is recorded.

4.2.11 Data-collection pipeline

The implemented data path is

two GELLO leaders → two FR3 follower controllers → ROS 2 bridge
 → assembled packets
 → LeRobot recorder → Parquet records and three RGB videos.

The follower-side system provides measured joint positions and current gripper widths. The command side provides absolute arm targets and absolute gripper-width commands. The recorder samples the assembled packet stream at 5 Hz, assigns frames to the active episode, and preserves the task identity. A complete episode contains a continuous visual and proprioceptive record from the initial condition to either task completion or termination.

Collection and task review are intentionally separate operations. The recorder verifies schema and continuity, while the operator verifies whether the physical task was demonstrated correctly. A numerically complete episode can still be unsuitable for training if the operator intervened incorrectly, the object left the intended workspace, or the terminal state does not match the language instruction.

4.2.12 Packet synchronization and action-label alignment

The stored cadence is 5 Hz, corresponding to approximately 0.2 s per step. The recorder pairs adjacent assembled packets. For the saved sample at index t ,

$$\mathbf{o}_t^{\text{visual}} = \text{three images from packet } t, \quad (4-12)$$

$$\mathbf{s}_t = \text{bilateral measured state from packet } t, \quad (4-13)$$

$$\mathbf{a}_t^{\text{arm}} = \mathbf{q}_{t+1}^{\text{target}} \text{ from packet } t + 1, \quad (4-14)$$

$$\mathbf{a}_t^{\text{grripper}} = \mathbf{g}_t^{\text{target}} \text{ from packet } t. \quad (4-15)$$

Images and measured state therefore come from the same current packet. The arm label is shifted forward by one packet so that the current observation is paired with the next absolute joint target. The gripper label follows the existing same-packet convention.

This procedure guarantees consistency of the saved software packet and label rule. It does not prove that all three camera exposures occurred at the same physical instant. The recorder does not interpolate device timestamps, perform ROS approximate-time alignment, or use a hardware trigger. It trusts the upstream bridge to provide an assembled packet. This limitation must be retained in the report because a nominal 5 Hz LeRobot timestamp alone cannot be used to reconstruct true camera skew or end-to-end latency.

4.2.13 Dataset construction and episode split

The three task datasets contain 80 episodes each. All three use random seed 1000 and the same validation episode indices:

$$\{17, 35, 58, 65, 66, 72, 74, 75\}.$$

The remaining 72 episodes per task are used for training. Splitting by episode prevents neighbouring frames from the same physical trajectory from appearing in both the training and validation sets.

Table 4–5 Episode-level dataset split for the three bimanual tasks

Task dataset	Episodes	Frames	Training split	Validation split
dual_ring	80	46,326	72 episodes / 41,410 frames	8 episodes / 4,916 frames
dual_socket	80	38,511	72 episodes / 34,655 frames	8 episodes / 3,856 frames
dual_ram	80	34,813	72 episodes / 31,675 frames	8 episodes / 3,138 frames
Total	240	119,650	216 episodes / 107,740 frames	24 episodes / 11,910 frames

The tasks are sampled with equal probability during training. Task selection occurs before sample selection, so dual_ring does not receive a larger task-level probability merely

because it contains more frames. This choice balances the three instructions and prevents the longest average trajectory from dominating the gradient updates.

4.2.14 Dataset inclusion boundary and language-prompt contract

The final checkpoint is a three-dataset model. A fourth local directory, `dual_socket2`, contains the same LeRobot Dataset v3.0 feature schema and 80 episodes, but it is not enumerated by the training program used for the reported run. It therefore contributes no frame to the normalizer, sampler, optimizer updates, validation loss, checkpoint selection, or physical result. Schema compatibility is not evidence of use. Table 4–6 records this boundary explicitly.

Table 4–6 Dataset and instruction scope of the deployed $\pi_{0.5}$ checkpoint

Directory	Episodes	Used	Training instruction or exclusion
<code>dual_ring</code>	80	Yes	First, push the can to the middle, then pick up the black ring with the handle, put it over the can, and set it down.
<code>dual_socket</code>	80	Yes	Push the box with the power socket to the middle, then pick up the plug and insert it into the socket.
<code>dual_ram</code>	80	Yes	Pull the computer motherboard closer, then remove the memory module and place it in the yellow tray.
<code>dual_socket2</code>	80	No	Compatible local data directory, but absent from the task-prompt map and from the reported training, statistics, validation, and checkpoint.

The raw `tasks.parquet` metadata in each used directory contains the generic source task `franka_gello_teleop`. During loading, `PromptDataset` replaces that generic value with the task-specific instruction in Table 4–6. The prompt is consequently part of the executable training configuration rather than an informal caption added after data collection. Deployment uses the same instruction wording.

If `dual_socket2` is added in a later experiment, its statistical and sampling role must be decided before training. Treating four directories as four equal sampler groups would give the two socket directories a combined 50% task probability. Treating them as two sources of one semantic socket task would instead require a hierarchical sampler so that ring, socket, and RAM remain approximately one third each. Either choice also requires new normalization statistics, validation reporting, checkpoint selection, and physical evaluation; the present 65,000-step result cannot be re-labelled as a four-dataset experiment.

4.2.15 Training statistics and task-balanced sampling

The action and state normalizers are calculated from the 107,740 training frames obtained after the episode split. For every real state and action dimension, the implementation records the count, minimum, maximum, mean, standard deviation, and the 1st, 10th, 50th, 90th, and 99th percentiles. State and action use quantile normalization, while images use the identity numeric normalizer after pixel preprocessing.

For a scalar feature x_d in dimension d , the implemented quantile mapping can be represented by

$$\tilde{x}_d = 2 \cdot \text{clip} \left(\frac{x_d - Q_{0.01,d}}{Q_{0.99,d} - Q_{0.01,d}}, 0, 1 \right) - 1, \quad (4-16)$$

where $Q_{0.01,d}$ and $Q_{0.99,d}$ are computed from training frames only. The postprocessor applies the corresponding inverse scaling to predicted actions. Extreme values outside the stored quantile range are clipped for the model representation; the robot adapter still performs independent finite, joint, gripper, and workspace checks after unnormalization.

The normalizer pools frames, but the sampler balances tasks. If N_k is the number of training frames in task k , each sample in that task receives weight

$$w_{k,i} = \frac{1}{N_k}. \quad (4-17)$$

The total weight of each task is then $\sum_{i=1}^{N_k} w_{k,i} = 1$. With replacement enabled, the expected long-run sampler share is one third for ring, socket, and RAM, even though their frame counts differ. At batch size 4, 100,000 optimizer steps process approximately 400,000 sampled training items; the expected count is therefore about 133,000 items per task, subject to sampling variation.

This distinction matters. Quantile statistics are influenced by the number of frames in each directory, so the longer ring trajectories contribute more values to the pooled distribution. Gradient sampling, by contrast, gives the three semantic tasks equal expected probability. The design protects task balance at the optimizer interface without claiming that the normalization statistics are task-equal.

For offline validation, the program fixes at most 64 batches per task. With batch size 4, each validation point uses at most 256 frames from each task, selected reproducibly at program start. The resulting per-task loss and their unweighted macro-average are suitable for

comparing checkpoints under this fixed procedure. They do not summarize every validation frame, and they do not measure real-robot task completion.

4.2.16 $\pi_{0.5}$ base model and software configuration

The final controller uses the official `pi05_base` model. The upstream checkpoint is identified by `gs://openpi-assets/checkpoints/pi05_base/params`, and training reads the converted PyTorch parameters from `real-exp/.hf-cache/pi05_base_pytorch/model.safetensors`. The base architecture combines a PaliGemma `gemma_2b` vision-language component with a `gemma_300m` action expert. The model weights use `bfloat16`.

The project uses LeRobot 0.5.2 with local project modifications. The base commit is

`dc06dba9d6220ab99c94e83ec7f0d45a35b44bed`.

The dataset format is LeRobot Dataset v3.0. Training is executed on physical GPU 1 of an NVIDIA RTX A6000 with 48 GB of memory.

4.2.17 Model input construction and state–language representation

The base configuration has a model-side action dimension of 32 and an action horizon of 50. Project state and action are only 16-dimensional, so the preprocessor pads entries 16–31 with zeros and preserves the physical values in entries 0–15. Loss is evaluated only over the real 16 dimensions, and the deployment adapter removes the padded tail before any robot command is formed.

The numeric state is quantile-normalized to approximately $[-1, 1]$, then discretized into 256 bins. The resulting state tokens are placed beside the task instruction in the model prompt:

```
Task: <task instruction>, State: <discretized state tokens>;
Action:
```

The PaliGemma SentencePiece tokenizer uses right padding and a maximum sequence length of 200 tokens. This representation makes the task instruction and current bilateral state jointly available to the model, but the final physical experiment does not isolate their individual effects. Because each visually distinct scene was paired with its expected instruction, the evidence supports a language-conditioned unified interface rather than a claim that language alone determines the selected behavior.

The three camera images are supplied in the fixed order $\{\text{cam_left}, \text{cam_front}, \text{cam_right}\}$. Each image is resized and padded by the image-preprocessing pipeline described above, then mapped from $[0, 1]$ to $[-1, 1]$. No camera slot is empty in the final configuration. The preprocessor rejects a missing feature name rather than silently replacing one of the three deployed views.

4.2.18 Flow-matching training objective

The action expert is trained with a conditional flow-matching objective. Let $\mathbf{a} \in \mathbb{R}^{50 \times 16}$ be the normalized demonstrated action chunk and let $\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ be an equally shaped Gaussian-noise sample. A time value is sampled from

$$t_0 \sim \text{Beta}(1.5, 1.0), \quad t = 0.999t_0 + 0.001.$$

The interpolated action state is

$$\mathbf{x}_t = t\boldsymbol{\epsilon} + (1 - t)\mathbf{a}, \quad (4-18)$$

and the target velocity is

$$\mathbf{u}_t = \boldsymbol{\epsilon} - \mathbf{a}. \quad (4-19)$$

Conditioned on the three images, task/state tokens, and t , the action expert predicts $\mathbf{v}_\theta(\mathbf{x}_t, t)$. The implemented loss is

$$\mathcal{L}_{\text{FM}} = \frac{1}{B \cdot 50 \cdot 16} \sum_{b=1}^B \sum_{\tau=1}^{50} \sum_{d=1}^{16} (u_{b,\tau,d} - v_{\theta,b,\tau,d})^2. \quad (4-20)$$

Padding dimensions are not used to improve the reported value. The loss is calculated in normalized action space and is therefore dimensionless. It is not joint error in radians, gripper-width error in metres, Cartesian pose error, insertion error, or task success. This distinction controls the checkpoint discussion in Chapter 5.

4.2.19 Adaptation scope and trainable parameters

The final run uses action-expert-only adaptation. The entire PaliGemma vision-language component, including the SigLIP visual encoder, remains frozen. Trainable names are restricted to the Gemma action expert and the action/time projection modules. No LoRA or other PEFT adapter is used in this LeRobot experiment.

Table 4–7 Parameter and optimization scope of the final LeRobot $\pi_{0.5}$ run

Item	Implemented scope
Frozen parameters	3,449,982,704 parameters in the PaliGemma vision-language component
Trainable parameters	693,422,112 parameters in the action expert and action/time projections
Total parameters	4,143,404,816
Parameter-efficient adapter	None; parameter names containing LoRA are rejected
Image augmentation	Disabled for the reported run
Gradient checkpointing	Disabled
Automatic mixed precision	Disabled; model weights use <code>bfloat16</code> as configured
Compilation	<code>torch.compile</code> disabled
Relative-action conversion	Disabled in preprocessing and postprocessing

The parameter count clarifies what “fine-tuning” means in this report. The final controller is not trained from random initialization and the full vision-language backbone is not updated. The experiment adapts the action generation path to the project camera/state/action contract while preserving the base representation. A separate OpenPI/JAX socket experiment contains LoRA parameters and an eight-dimensional single-arm state/action record, but it is not the source of the deployed dual-arm checkpoint and is excluded from the final design claim.

4.2.20 Training procedure

The three tasks are trained jointly for 100,000 optimizer steps. Batch size is 4 and the optimizer is AdamW. The peak learning rate is 2.5×10^{-5} . A 1,000-step warm-up is followed by cosine decay to 2.5×10^{-6} . Task-balanced validation is run every 5,000 steps.

The training configuration deliberately separates task balancing from frame count. The action and state normalizers are derived from the training data only. Validation episodes are not used to compute model updates. The same action representation and `use_relative_actions=false` setting are retained when the selected checkpoint is loaded for deployment.

Table 4–8 Final $\pi_{0.5}$ training configuration

Parameter	Implemented value
Base model	Official <code>pi05_base</code>
Model structure	PaliGemma <code>gemma_2b</code> and action expert <code>gemma_300m</code>
Weight precision	<code>bfloat16</code>
LeRobot version	0.5.2, with local modifications
Dataset format	LeRobot Dataset v3.0
Training hardware	NVIDIA RTX A6000 48 GB, physical GPU 1
Batch size	4
Optimizer	AdamW
Training length	100,000 joint-training steps
Learning-rate schedule	1,000-step warm-up to 2.5×10^{-5} , followed by cosine decay to 2.5×10^{-6}
Task sampling	Equal probability for <code>dual_ring</code> , <code>dual_socket</code> , and <code>dual_ram</code>
Validation interval	Every 5,000 optimizer steps

4.2.21 Optimizer schedule, runtime, and checkpoint package

AdamW uses $\beta_1 = 0.9$, $\beta_2 = 0.95$, $\epsilon = 10^{-8}$, weight decay 0.01, and global gradient-norm clipping at 1.0. During the first 1,000 optimizer steps, the learning rate increases to 2.5×10^{-5} . Cosine decay then reduces it to 2.5×10^{-6} at step 100,000. One iteration executes preprocessing, gradient reset, forward loss calculation, backpropagation, gradient clipping, optimizer update, and scheduler update in that order.

Each checkpoint contains the policy configuration, complete model weights, preprocessor and postprocessor definitions, quantile normalizers, the PaliGemma tokenizer asset, the multi-task manifest, optimizer state, scheduler state, random-number-generator state, and training step. The early processor configuration used an absolute tokenizer path. A portability repair copied the tokenizer model into each checkpoint and rewrote the processor reference to a relative file. The watcher verified the repair through step 100,000.

Saving complete model and optimizer states supports continuation and auditing, but it has a substantial storage cost. It also does not, by itself, make the experiment fully reproducible: model artifacts preserve the trained state, whereas source, environment, data identity, and base-weight provenance determine whether the state can be recreated.

Table 4–9 Measured training resource use and checkpoint storage

Resource or artifact	Recorded value
Complete-run elapsed time	17 h 01 min 17 s
Typical optimizer-step time	Approximately 0.60 s at batch size 4
Maximum allocated GPU memory	23.49 GiB
Final training-window loss	0.028063; not the checkpoint-selection metric
Checkpoint interval	Every 5,000 steps from 5,000 through 100,000
Number of complete checkpoints	20
Approximate output-directory size	216 GB
Model file per checkpoint	Approximately 9.35 GB
Optimizer state per checkpoint	Approximately 2.20 GB
Selected checkpoint	Step 65,000, chosen by held-out macro loss

4.2.22 Validation and checkpoint selection

For each task, validation first computes the mean normalized flow-matching loss over at most 64 batches. With batch size 4, this corresponds to at most 256 validation samples per task at one validation point. The checkpoint-selection quantity is the unweighted task macro-average

$$\mathcal{L}_{\text{macro}} = \frac{\mathcal{L}_{\text{ring}} + \mathcal{L}_{\text{socket}} + \mathcal{L}_{\text{ram}}}{3}. \quad (4-21)$$

The best checkpoint is the validation point with the lowest $\mathcal{L}_{\text{macro}}$.

This metric is intentionally not weighted by the number of validation frames. It gives each task equal influence, just as the training sampler gives each task equal selection probability. The loss supports checkpoint comparison, but it is not a physical success rate, insertion-depth measurement, contact-force measurement, or calibrated pose error. Real-robot performance is evaluated separately in Chapter 5.

4.2.23 Action chunking and temporal interface

Collection and execution use 5 Hz, so one action step represents 0.2 s. The model predicts a 50-step chunk:

$$T_{\text{predicted}} = \frac{50}{5} = 10 \text{ s}. \quad (4-22)$$

Deployment uses $n_action_steps=10$, so the executor consumes approximately

$$T_{\text{executed}} = \frac{10}{5} = 2 \text{ s} \quad (4-23)$$

before requesting a new policy result from updated observations.

This is a receding-horizon interface. The 50-step prediction gives the action expert a longer temporal target, while executing only 10 steps limits the amount of open-loop motion. The exact policy-query interval may also include capture, preprocessing, inference, transfer, and queue overhead; therefore, the nominal two-second value describes the consumed action segment rather than a measured end-to-end latency.

4.2.24 Deployment pipeline

Deployment begins by loading the selected unified checkpoint and its associated normalization statistics. The system verifies the three camera feature names, checks the 16-dimensional state, and confirms the language instruction. The current observation is preprocessed and passed to the policy server. The returned action chunk is denormalized, sliced to the 16 valid robot dimensions, checked for finite values and limits, and divided into left-arm, left-gripper, right-arm, and right-gripper targets.

The robot adapter sends absolute joint targets and absolute gripper widths. It does not reinterpret them as deltas. After the configured ten-step segment, new images and state are acquired and a new action chunk is requested. This loop continues until the terminal task condition or a stop condition is reached.

4.2.25 Control, computation, and safety boundary

The policy is not the safety controller. Before motion is enabled, a dry run verifies camera freshness and ordering, state/action dimensions, normalization statistics, checkpoint identity, language instruction, finite model outputs, network communication, joint and gripper limits, workspace bounds, and emergency-stop access.

During motion, the run stops on a missing image, stale state, malformed packet, non-finite output, excessive target, workspace violation, communication timeout, robot protective stop, emergency stop, or manual stop. Reset is performed only after policy execution has stopped and the shared workspace is clear. The power socket remains unpowered. These checks maintain a deterministic boundary around the probabilistic learned policy.

4.3 Detailed system design

4.3.1 Workcell layout

The two FR3 arms are mounted upright on opposite sides of a shared tabletop region. Task fixtures are placed inside the verified overlap of the two practical workspaces. The D435i is fixed above the shared region. One D405 is attached to each wrist. Each arm carries one custom gripper. The two GELLO leaders remain outside the follower workcell and are used by the operator during demonstration collection.

Figure 4–1 identifies the three deployed camera views and the 16-dimensional bilateral state supplied to the policy, but it is a model-level architecture diagram rather than a dimensioned workcell drawing. Exact base spacing, camera height, camera pitch, and fixture coordinates were not included in the supplied implementation record; these values should be added to a future dimensioned workcell drawing rather than inferred from the conceptual image.

4.3.2 Dual-arm robotic platform

Both arms use the same 16-value bilateral ordering defined in Equations 4–7 and 4–8. The left and right controllers remain independent low-level systems, but they are driven from one policy output and one episode timeline. Each arm can perform a large scene-preparation motion or a precise manipulation motion, depending on the demonstrated task role.

The redundant seventh joint is important in the shared workcell because end-effector pose alone does not determine a unique arm posture. Demonstrations implicitly select postures that provide camera visibility, avoid fixtures, and reduce inter-arm interference. The low-level controller remains responsible for executing the admissible targets; the learned model does not directly output torque.

4.3.3 GELLO leader assemblies

Each leader measures seven arm channels and one gripper-related channel for its assigned follower. The operator does not bear follower-arm load through the leader; it is a position-input and command device. Mechanical looseness, servo zero error, cable interference, or incorrect joint signs can degrade demonstration quality, so the two leader assemblies are inspected and checked independently before bilateral collection. The component-level con-

struction and quantities are recorded in Appendix 2, rather than repeated in the system-level design description.

4.3.4 End-effector and mechanical components

Each gripper uses two screw-mounted compliant fingers. The mounting screws must be tightened consistently, and the fingers must be installed with matching orientation. Before a run, the operator checks for tearing at rib roots, permanent bending, loose mounting holes, surface contamination, and insufficient opening range.

The task fixtures prevent uncontrolled object drift while retaining the manipulation required by the task. The can and socket box must still be pushable into the intended central region. The motherboard must be graspable and movable by the left gripper but sufficiently supported to allow the right gripper to extract the memory module. The tray must remain fixed so that successful placement is not confused with tray motion.

4.3.5 Vision and sensing system

The D435i mount is fixed relative to the workcell and is positioned to keep all source and destination regions in view. The two D405 mounts move with the wrists and are oriented so that the gripper contact region appears in the close-range image during approach. Cables are routed to avoid entering the gripper opening or restricting wrist motion.

The logical camera mapping is fixed:

$$\begin{aligned} \text{overhead} &\equiv \text{D435i}, \\ \text{left_wrist} &\equiv \text{left D405}, & \text{right_wrist} &\equiv \text{right D405}. \end{aligned}$$

The recorder and policy use these identities consistently. The three cameras provide RGB to the model; their depth and inertial capabilities are not part of the current learned observation.

4.3.6 Demonstration data-collection system

The collection system starts the two robot interfaces, both gripper interfaces, three camera publishers, the ROS 2 bridge, and the LeRobot recorder. The bridge assembles the latest required fields into a packet. The recorder begins a named task episode only after all required fields are available.

The operator executes the complete instruction and then ends the episode. The episode is retained only if the three image streams decode correctly, state and action arrays have the required width, and the demonstration matches the intended task. Manual workcell reset is performed between episodes. The same task instruction is used during collection and later deployment so that language conditioning does not depend on an undocumented paraphrase.

4.3.7 Data processing pipeline

Processing has separate visual and numeric branches. The visual branch decodes the three RGB streams, converts them to channel-first tensors, preserves aspect ratio during resizing, pads to 224×224 , and normalizes to $[-1, 1]$. The numeric branch reads the 16-dimensional state and action, applies the training statistics, and pads the model-side representation to 32 dimensions.

Episode metadata determines the task and split. The eight validation episode indices are excluded from training for every task. During data loading, the task-balanced sampler chooses among the three task datasets with equal probability. The language instruction, three images, state, and supervised action target are then passed to the policy training step.

4.3.8 Policy-training system

The training system loads `pi05_base`, the converted PyTorch weights, and the LeRobot v3.0 datasets. It uses batch size 4 on the RTX A6000. Optimizer state, learning-rate schedule, model state, step index, and validation quantities are saved at the configured checkpoints.

Validation runs every 5,000 steps. Each task is evaluated separately before the unweighted macro-average is calculated. This preserves per-task traceability and prevents a low loss on the largest dataset from concealing a poor result on another task.

4.3.9 Robot control and deployment system

The deployment system loads one checkpoint for all three instructions. The selected instruction and current observation are sent to $\pi_{0.5}$. A 50-step action chunk is returned, and the executor uses the first ten steps. The target ordering, absolute-action interpretation, 5 Hz cadence, and camera mapping are identical to training.

The left and right arm targets are separated only after policy output validation. The grip-

per commands are expressed as absolute opening widths in metres. After each executed segment, the queue is not blindly extended with the rest of the old 50-step chunk; the policy is queried again using updated images and state.

4.3.10 Hardware–software interfaces

Table 4–10 Detailed hardware–software interfaces in the final system

Information	Source	Destination	Implemented contract
Left leader joints	Left GELLO	Left follower interface	Seven leader positions mapped to left FR3 joint order, signs, offsets, and permitted range.
Left leader gripper	Left GELLO	Left gripper interface	One leader channel mapped to absolute gripper opening width.
Right leader joints	Right GELLO	Right follower interface	Seven leader positions mapped to right FR3 joint order, signs, offsets, and permitted range.
Right leader gripper	Right GELLO	Right gripper interface	One leader channel mapped to absolute gripper opening width.
Left measured joints	Left FR3 controller	ROS 2 bridge / recorder / executor	Seven current joint positions in state indices 0–6.
Left measured gripper	Left gripper	ROS 2 bridge / recorder / executor	Current opening width in metres at state index 7.
Right measured joints	Right FR3 controller	ROS 2 bridge / recorder / executor	Seven current joint positions in state indices 8–14.
Right measured gripper	Right gripper	ROS 2 bridge / recorder / executor	Current opening width in metres at state index 15.
Overhead RGB	RealSense D435i	Bridge / recorder / policy	RGB 424×240 at 5 Hz; stored as $3 \times 240 \times 424$, then resized and padded to $3 \times 224 \times 224$.
Left-wrist RGB	Left RealSense D405	Bridge / recorder / policy	Same tensor contract under the fixed left-wrist feature name.
Right-wrist RGB	Right RealSense D405	Bridge / recorder / policy	Same tensor contract under the fixed right-wrist feature name.
Assembled packet	ROS 2 bridge	LeRobot recorder	Same-packet images and measured state; adjacent-packet arm action label.

Information	Source	Destination	Implemented contract
Episode record	LeRobot recorder	Dataset v3.0	Three videos, state, absolute action, task instruction, episode and frame indexing.
Training sample	Dataset and sampler	$\pi_{0.5}$ trainer	Equal-probability task sampling; training episodes only.
Language instruction	Task interface	$\pi_{0.5}$ policy	One of the three documented task instructions matching the prepared scene.
Policy observation	Preprocessor	$\pi_{0.5}$ policy	Three normalized $3 \times 224 \times 224$ images and a padded numeric state whose first 16 entries are valid.
Predicted action	$\pi_{0.5}$ policy	Robot adapter	50-step chunk; first 10 steps consumed per query.
Left arm command	Robot adapter	Left FR3 controller	Absolute joint targets from action indices 0–6.
Left gripper command	Robot adapter	Left gripper	Absolute opening width from action index 7.
Right arm command	Robot adapter	Right FR3 controller	Absolute joint targets from action indices 8–14.
Right gripper command	Robot adapter	Right gripper	Absolute opening width from action index 15.
Stop conditions	Cameras, bridge, adapter, robot controllers, operator	Both robot command paths	Stop on stale/missing data, invalid action, timeout, workspace or joint violation, protective stop, emergency stop, or manual stop.

4.3.11 Task definitions and bilateral role allocation

4.3.12 System operation

4.3.12.1 Pre-run checks

The operator confirms the checkpoint, instruction, object arrangement, camera mapping, gripper condition, network status, and robot status. Both arms are returned to the approved start region. The socket is verified to be unpowered. The shared workspace is cleared of unrelated objects, and the emergency stop remains accessible.

Table 4–11 Language instructions, bilateral roles, and terminal conditions

Task	Language instruction	Demonstrated arm roles	Terminal completion condition
T1	First, push the can to the middle, then pick up the black ring with the handle, put it over the can, and set it down.	Left arm pushes the can; right arm grasps the handled ring and places it.	The can is repositioned and the released ring remains stably over the can.
T2	Push the box with the power socket to the middle, then pick up the plug and insert it into the socket.	Right arm pushes the socket box; left arm grasps and inserts the plug.	The socket box is repositioned and the plug remains inserted without manual assistance.
T3	Pull the computer motherboard closer, then remove the memory module and place it in the yellow tray.	Left arm grips and pulls the motherboard; right arm removes and places the memory module.	The memory module is fully removed and remains inside the yellow tray.

4.3.12.2 Collection run

The leader offsets and channel directions are checked. Recording begins only after all three cameras and all bilateral state/action fields are present. The operator executes the complete task through the two GELLO leaders. The episode is stopped, reviewed, accepted or rejected, and then the scene is reset.

4.3.12.3 Policy dry run

The policy is first queried without enabling robot motion. The returned shape, finite values, absolute-action interpretation, camera ordering, and initial target relative to the current robot state are inspected. A dry run that fails any check is not promoted to physical execution.

4.3.12.4 Autonomous run

The executor enables the approved command path, consumes ten actions from each predicted chunk, and re-observes the workcell. The operator does not manually assist a counted trial. The trial ends when the documented terminal condition is met or a stop condition occurs. Outcome, stop reason, checkpoint, instruction, and video record are retained for Chapter 5 evaluation.

4.3.13 Parts and components

Table 4–12 retains only the system-level composition needed to understand the implementation. Manufacturer model details, leader subcomponents, and the complete 22-item parts record are provided in Appendix 2, Table 2–5.

Table 4–12 System-level component summary

System group	Quantity	Function
FR3 follower systems	2	Left- and right-arm manipulation with low-level robot control and protection.
Custom grippers	2	Task-compatible grasping with four compliant TPU fingers in total.
RGB camera views	3	One fixed overhead view and two wrist-local views.
GELLO leaders	2	Bilateral demonstration input, one leader per follower arm.
Training and bridge compute	1 system	Dataset recording, packet assembly, policy training, validation, and deployment.
Task-object and fixture sets	3	Ring/can, plug/socket, and motherboard/memory-module tasks.
Safety and storage resources	1 set	Deterministic stopping, supervision, dataset storage, checkpoints, and logs.

4.4 Prototype manufacturing and implementation

4.4.1 Implementation scope

The project reuses laboratory robot arms, controllers, cameras, computing resources, and network infrastructure. Custom manufacturing is concentrated in the compliant fingers, GELLO structural parts, camera mounts, and task fixtures. Software implementation covers collection, packet assembly, LeRobot conversion, preprocessing, multi-task sampling, $\pi_{0.5}$ training, validation, inference, action adaptation, and run logging.

The full economic value of the workcell is not equal to the incremental cash expenditure. The FR3 arms, cameras, GPU, and safety equipment are in-kind laboratory resources. The DR3 budget recorded no additional out-of-pocket expense for the proof of concept. The expanded budget below preserves that statement while separating unpriced in-kind assets

from incremental fabrication cost.

4.4.2 Materials and manufacturing processes

Table 4–13 Materials, manufacturing processes, and critical considerations

No.	Part or resource	Material or source	Process or tool	Critical consideration
1	FR3 arms and controllers	Laboratory equipment	Upright installation and interface configuration	Base rigidity, shared workspace, controller limits, network isolation, emergency stop
2	Compliant fingers	TPU 64D	Fused-filament 3D printing, support removal, inspection	Print orientation, rib integrity, screw-hole quality, paired geometry, permanent deformation
3	Gripper attachment	Existing gripper and printed/adapted interfaces	Mechanical assembly with screws	Alignment, thread engagement, consistent tightening, opening clearance
4	GELLO structures	FPX330-H101 and printed parts	Additive manufacturing and modular assembly	Joint-axis alignment, low friction, cable strain relief, repeatable zero offsets
5	Camera mounts	Laboratory or printed brackets	Rigid mounting and cable routing	Working distance, FOV, gripper visibility, occlusion, cable interference
6	Task fixtures	Printed or manually assembled fixture parts	Additive manufacturing and bench assembly	Repeatable initial pose, pushability where required, no mains power, stable destination
7	Software environment	LeRobot, PyTorch, ROS 2, project modifications	Version-controlled configuration and scripted execution	Commit traceability, feature names, absolute-action setting, dataset/model compatibility
8	Dataset and checkpoints	Recorded demonstrations and training outputs	Validation, indexing, backup, and integrity checks	Episode-level split, three-view completeness, task balance, reproducible checkpoint selection

4.4.3 Gripper fabrication and inspection

The finger CAD is prepared for TPU 64D printing with the mounting face and rib structure preserved. The selected print orientation must avoid weak inter-layer loading at the mounting holes and rib roots. After printing, supports and stringing are removed without cutting the

compliant ribs. The overall envelope, paired symmetry, hole condition, and free deformation are inspected before assembly.

The two fingers on one gripper are installed with matching orientation and adequate clearance through the commanded opening range. A manual opening-and-closing check verifies that neither finger contacts the camera mount or wrist cable. The assembly is rejected if a rib is torn, the base is permanently warped, a screw cannot retain the finger, or the two contact surfaces are visibly mismatched.

4.4.4 GELLO assembly and calibration

Each leader is assembled from the FPX330-H101 structure, printed links and brackets, seven M288 actuators, one M077 actuator, controller board, and 5 V power supply. Joint axes are checked for free movement before wiring. Cables are routed so that the full demonstrated range does not pull on connectors or bias the leader posture.

After electrical assembly, servo identifiers, zero positions, directions, and range mappings are verified one channel at a time. The left and right leaders are calibrated independently because a mirrored workcell does not imply identical signs and offsets. The final mapping is stored with the collection configuration so that later episodes can be traced to the leader calibration used.

4.4.5 Camera and fixture implementation

The overhead mount is installed so that the complete source, interaction, and destination regions remain visible. Each D405 is mounted within its intended 7–50 cm close-range operating envelope during manipulation. The wrist view is checked at approach, contact, and retreat postures, not only at the robot start pose.

Task fixtures are placed in the verified shared workspace. The can, socket box, and motherboard must permit the instructed preparation motion, while their target regions remain visible. The yellow tray is fixed against accidental displacement. The socket fixture is electrically isolated and unpowered. Reference markings or stops are used where necessary to make scene reset repeatable.

4.4.6 Software implementation

The software stack is configured in independent stages:

1. robot, gripper, camera, and GELLO device interfaces;
2. ROS 2 state and command topics;
3. bridge packet assembly and freshness checks;
4. LeRobot Dataset v3.0 episode recording;
5. image preprocessing and 16-to-32-dimensional model padding;
6. episode-level split and equal-probability task sampling;
7. $\pi_{0.5}$ fine-tuning and macro-loss validation;
8. unified-checkpoint policy inference; and
9. bilateral absolute-action adaptation and safety checks.

The configuration records LeRobot version 0.5.2, the project commit, local modifications, base-checkpoint source, training steps, learning-rate schedule, camera feature names, `use_relative_actions=false`, action chunk size, and `n_action_steps`. These values are treated as part of the reproducible implementation rather than left as implicit code defaults.

4.4.7 System integration sequence

Integration proceeds from independent hardware checks to full learned control. First, each arm and gripper is checked without the policy. Second, all three cameras are checked under their final logical names. Third, each GELLO leader is mapped to its follower. Fourth, bilateral teleoperation is tested at low speed with an empty workspace. Fifth, complete episodes are recorded and decoded. Sixth, a training batch is inspected for camera order, numeric order, and action alignment. Seventh, the policy is run without opening the robot command path. Finally, supervised physical trials are enabled.

This sequence isolates failures. A camera-identity error should be found before training; a leader offset should be found before collecting a large dataset; a state/action index error should be found before robot execution; and a policy-output error should be found during the dry run.

4.4.8 Budget

Table 4–14 separates the reported incremental cash cost from the laboratory equipment, compute, and fabrication resources supplied in kind. This distinction avoids presenting an in-kind research workcell as a zero-value system.

Table 4–14 Expanded implementation budget and resource status

No.	Component, service, and quantity	Provider	Cash cost	Status and accounting note
1	Two FR3 arms, controllers, and safety equipment (2 sets)	Laboratory	CNY 0 incremental	Existing in-kind assets; acquisition value not supplied
2	D435i, two D405 cameras, and mounts (3 cameras)	Laboratory	CNY 0 incremental	Existing in-kind assets; mount material not separately recorded
3	RTX A6000 training workstation and storage (1 set)	Laboratory	CNY 0 incremental	Existing shared compute; energy and depreciation not allocated
4	GELLO leader assemblies (2)	Laboratory / project resources	CNY 0 incremental	DR3 records availability; itemized acquisition value not supplied
5	TPU 64D compliant fingers (4)	Student Innovation Center	CNY 0 incremental	Fabricated through available resources; material mass and print time not recorded
6	Task objects and fixtures (3 sets)	Laboratory / Student Innovation Center	CNY 0 incremental	Available; final fixture-material value not supplied
7	Software stack (1 configuration)	Open source and project code	CNY 0 licence cost	LeRobot, ROS 2, PyTorch, and project modifications
Reported additional out-of-pocket cost			CNY 0	Excludes in-kind asset value, labour, energy, and depreciation

The CNY 0 value is therefore an incremental cash figure, not the total system value. A full economic budget would require the purchase or replacement value of two FR3 systems, two GELLO assemblies, three cameras, the GPU workstation, storage, safety hardware, print material, print time, labour, and energy. Those values were not included in the supplied DR3

or final implementation record and should be added only from invoices or the laboratory asset register.

4.4.9 Software, environment, and checkpoint provenance

The project records Git HEAD=dc06dba9d6220ab99c94e83ec7f0d45a35b44bed, but that identifier is only the base repository state. At inspection time, the index contained approximately 705 staged deletions, the current project files appeared again as untracked files, and the final $\pi_{0.5}$ training program was itself untracked. The commit therefore cannot be cited as a complete source snapshot for the 100,000-step run.

To make the inspected implementation identifiable even without a clean commit, Table 4–15 records hashes for the training path that produced the final experiment.

Table 4–15 SHA-256 identifiers for key $\pi_{0.5}$ implementation files

Project-relative file	SHA–256
train/train_lerobot_pi05_multitask.py	83ad1d9b4319dc225701cb8f236cbfea77b6d7d61ebfc7d0b
scripts/train_dual_pi05_multitask.py	05c717449929362d84f1b1068c97fd02939bc7a71616fc132
scripts/train_dual_pi05_multitask.py	6d862671837ff11c1d2c50f9f76549eab2afbfb606e98883
configuration_pi05.py	c66dfddc27557dd89a1cabd1354f041923bfcfb541d348185a
modeling_pi05.py	263a2c1e40e5a1cf0ab0808f149a1a383b337c0ace4bddff1
processor_pi05.py	ca9cca8d54a681e13072b3acfb557f7b8e77c37ac4452bdbc

The observed training environment used Python 3.12.13, LeRobot 0.5.2, PyTorch 2.10.0+cu128, and Transformers 5.3.0. The supplied `environment.yml` does not lock this complete stack. The converted base weights are present locally, but the download identifier and conversion command were not retained with the run. The final reproducibility claim is consequently partial: the checkpoint and key files are identifiable and deployable, but recreating the same training state from the repository alone is not yet guaranteed.

4.4.10 Reproduction checklist for the final configuration

A future release should treat the physical and virtual assets as one versioned prototype package. At minimum, reproduction of the final configuration requires the following checks:

1. archive the exact hashed training, processor, executor, and deployment files under one clean commit;

2. export a complete environment lock including CUDA-dependent packages;
3. record the upstream base-checkpoint identifier and conversion command;
4. preserve the three used dataset versions, prompts, episode indices, normalization statistics, and the explicit `dual_socket2` exclusion;
5. retain the step-65,000 checkpoint, its tokenizer, processors, multi-task manifest, and training state;
6. record camera identities, bilateral feature ordering, absolute-action convention, chunk settings, and robot-side limits;
7. perform a no-motion policy dry run before enabling either FR3 command path; and
8. reproduce the Chapter 5 trial definitions without using the reported outcomes to alter the checkpoint.

Items such as exact base poses, camera extrinsics, print settings, fixture coordinates, and measured latency remain necessary for a stronger reproduction package. They are listed as gaps rather than filled with inferred values.

4.4.11 Current implementation status and remaining documentation

The final implementation now includes dual-arm state and action handling, three RGB inputs, LeRobot Dataset v3.0 data, one jointly trained $\pi_{0.5}$ checkpoint, equal-probability three-task sampling, the defined episode-level validation split, macro flow-matching checkpoint selection, and a 5 Hz absolute-action deployment interface. The deployed artifact is the step-65,000 checkpoint; the later step-100,000 checkpoint is retained as the final training state but is not re-labelled as the validation optimum. The selected DR3 hardware modules and engineering drawings have been retained in this chapter.

Several quantities remain documentation gaps rather than model-design gaps. They include measured FR3 base poses, camera extrinsics and mounting distances, dimensioned fixture locations, ring and can clearances, and final socket insertion clearance and depth. They also include motherboard and memory-module clearances, measured gripper force–deflection behaviour, TPU print settings, print mass and time, full in-kind asset value, and measured end-to-end latency. These values should be added if the report is expected to make production-level tolerance, structural, timing, or economic claims.

Chapter 5 Validation results

5.1 Validation objectives and scope

The validation evaluates the final unified $\pi_{0.5}$ system described in Chapter 4. Its principal purpose is to determine whether one jointly trained checkpoint can execute the three documented bimanual tasks on the physical dual-FR3 workcell and whether the completed prototype meets the six engineering specifications defined in Chapter 2. The evaluation therefore separates three evidence families that have different meanings:

1. the offline normalized flow-matching loss used to select a checkpoint; and
2. the 25-trial planning, detection, pose-error, failure-detection, and recovery evaluations associated with ES1 and ES3–ES6; and
3. the end-to-end physical task success rate measured over 75 deployments for ES2.

The first quantity measures agreement with held-out demonstration samples and is used only for model selection. The second family evaluates defined intermediate engineering outcomes under separate 25-trial protocols. The third measures whether a complete physical trial reaches the task-specific terminal condition in Table 4–11. A low validation loss is not treated as a substitute for physical task completion, and a configured threshold is not treated as a measured result.

The physical ES2 evaluation covers the three bounded task configurations used during final implementation: ring placement over a can, insertion of a two-pin plug into a two-hole socket, and memory-module removal from a motherboard followed by placement in a yellow tray. Each task was evaluated in 25 counted trials, giving 75 physical trials in total. The ES1, ES3, ES4, ES5, and ES6 records each use 25 experiments under their own operational definitions. These populations are reported separately because their denominators and success events are not interchangeable. The results establish performance within the prepared laboratory workcell; they do not by themselves demonstrate open-world object generalization, arbitrary language understanding, certified safety, or production-line reliability.

5.2 Validation design and engineering-specification mapping

5.2.1 Measurement rules and independent denominators

Table 5–1 maps every design-stage specification to its final measurement rule. A rate is calculated from a binary outcome:

$$\hat{p} = \frac{x}{n}, \quad (5-1)$$

where x is the number of successful outcomes and n is the number of counted experiments for that specification. The project acceptance decision uses the observed point estimate $\hat{p}$ and the threshold defined in Table 2–2. Statistical intervals are reported later to show the uncertainty associated with small samples; they do not replace the project acceptance rule.

Table 5–1 Final validation protocol for engineering specifications ES1–ES6

ES	Metric and target	Counted success or measurement rule	Test count	Exclusion and boundary
ES1	Planning success > 90%	The task instruction, initialized scene, current observation, and loaded unified checkpoint form a valid executable plan before physical motion.	25	A rejected initialization is not counted as a later execution success. This metric evaluates executable-plan formation under the prepared scenes, not general autonomous planning.
ES2	Task execution > 70%	A complete autonomous run reaches the task-specific terminal condition in Table 4–11 without corrective assistance after start.	75 total; 25 per task	Partial completion and assisted completion are failures. Overall and per-task values are both retained.
ES3	Component detection > 90%	The required component for the active task is correctly identified in the evaluated observation.	25	The result applies to the documented objects and camera arrangement. It is not an arbitrary-object detection benchmark.

ES	Metric and target	Counted success or measurement rule	Test count	Exclusion and boundary
ES4	Pose error < 20 mm	The recorded component-position error is compared with the 20 mm limit. Successful executions are not entered as artificial zero-error observations.	25 experiments; recorded-error subset	The available record states that the measured errors are below 20 mm but does not provide the individual error series or the size of the recorded-error subset. No mean or variance is inferred.
ES5	Failure detection > 90%	An induced or naturally occurring failure is identified before an additional unsafe action is issued.	25	This is a project-level detection test around the learned policy, not a functional-safety certification.
ES6	Recovery success > 70%	The defined recovery procedure restores a valid task state after a detected failure.	25	A recovery opportunity is counted separately from the original ES2 task attempt and does not increase the ES2 numerator.

5.2.2 Observed specification-level outcomes

The observed outcomes are summarized before the detailed task results so that the final design can be judged against its original targets.

All six project-level point estimates or operational measurements satisfy their stated acceptance rule. This statement must be read together with two qualifications. First, ES2 is heterogeneous: the overall result passes because the RAM and ring tasks are strong, while socket insertion remains below 70%. Second, the rate tests contain only 25 observations each and ES4 does not include a supplied continuous error series. The results demonstrate compliance with the project decision rules under the evaluated conditions, not precise population parameters or production reliability.

Table 5–2 Engineering-specification compliance under the final validation protocols

ES	Target	Observed result	Point-estimate judgment	Interpretation
ES1	> 90%	23/25 = 92%	Pass	The observed executable-plan rate exceeds the project threshold.
ES2	> 70%	59/75 = 78.7%	Pass overall	The aggregate passes; the socket subtask is 52% and does not pass individually.
ES3	> 90%	24/25 = 96%	Pass	The observed detection rate exceeds the project threshold for the evaluated components.
ES4	< 20 mm	Recorded error < 20 mm	Pass under recorded-error rule	Successful executions were excluded rather than recorded as zero error; no distributional statistic is claimed.
ES5	> 90%	23/25 = 92%	Pass	The observed failure-detection rate exceeds the project threshold.
ES6	> 70%	23/25 = 92%	Pass	The observed recovery rate exceeds the project threshold under the defined recovery opportunities.

5.3 Checkpoint selection and deployment configuration

5.3.1 Selection criterion

Chapter 4 defines the checkpoint-selection procedure in Equation 4–21. Validation was performed every 5,000 optimizer steps. For each validation point, the normalized flow-matching losses of the ring, socket, and RAM datasets were first calculated separately and then combined as the unweighted macro-average

$$\mathcal{L}_{\text{macro}} = \frac{\mathcal{L}_{\text{ring}} + \mathcal{L}_{\text{socket}} + \mathcal{L}_{\text{ram}}}{3}.$$

The checkpoint with the lowest recorded $\mathcal{L}_{\text{macro}}$ was selected. Under this criterion, the best checkpoint occurred at 65,000 training steps, where the macro loss was 0.036552. The model selected for physical deployment was therefore the 65,000-step unified checkpoint rather than the final 100,000-step training state.

This selection preserves equal task importance. The ring dataset contains more frames than the other two datasets, but neither the training sampler nor the checkpoint-selection metric gives it greater task-level weight.

5.3.2 Validation trajectory

Figure 5–1 shows all 20 scheduled validation points. The largest decline occurs early in training: the macro loss decreases from 0.047246 at step 5,000 to 0.038788 at step 25,000. Later values fluctuate within a narrower range. Step 65,000 is the minimum recorded macro value even though individual tasks can attain lower loss at another checkpoint. For example, the socket loss at step 100,000 is slightly lower than its value at step 65,000, but the selection rule evaluates the three-task macro rather than optimizing one task in isolation.

Table 5–3 Recorded validation losses throughout $\pi_{0.5}$ training

Step	Macro	Ring	Socket	RAM
5,000	0.047246	0.047841	0.042100	0.051797
10,000	0.045191	0.043725	0.042415	0.049435
15,000	0.040952	0.042677	0.036072	0.044108
20,000	0.039637	0.039171	0.034430	0.045309
25,000	0.038788	0.038972	0.031825	0.045566
30,000	0.039298	0.038914	0.032771	0.046208

Step	Macro	Ring	Socket	RAM
35,000	0.037891	0.037872	0.034088	0.041713
40,000	0.039788	0.039263	0.036138	0.043962
45,000	0.039579	0.037413	0.036164	0.045160
50,000	0.038006	0.036331	0.033317	0.044369
55,000	0.038648	0.033924	0.032502	0.049518
60,000	0.038844	0.038564	0.033710	0.044257
65,000	0.036552	0.035224	0.031442	0.042989
70,000	0.038273	0.037573	0.031956	0.045290
75,000	0.037385	0.034964	0.033154	0.044036
80,000	0.038209	0.034621	0.032229	0.047779
85,000	0.039486	0.037060	0.033158	0.048240
90,000	0.037782	0.037074	0.032224	0.044047
95,000	0.037945	0.036334	0.033529	0.043973
100,000	0.037026	0.036607	0.031235	0.043237

The selected macro loss is approximately 22.6% lower than the value at step 5,000. Values after step 65,000 do not improve the recorded macro criterion. This observation justifies the selected checkpoint under the declared procedure, but it is not an early-stopping experiment and does not prove that later training causes physical overfitting. Physical behavior is measured only by the trials reported below.

5.3.3 Deployment hardware

Training was performed on the RTX A6000 system documented in Chapter 4. Physical deployment used an NVIDIA GeForce RTX 5070 Ti. The deployed policy used the same three image inputs, 16-dimensional bilateral state, absolute 16-dimensional action representation, language-instruction interface, and `use_relative_actions=false` setting as the training configuration. It predicted a 50-step action chunk and the executor consumed ten steps before requesting an updated prediction.

The deployment-GPU model is recorded to distinguish training compute from inference compute. A complete task execution took approximately one minute under the evaluated conditions. This is an approximate operational duration rather than a mean calculated from per-trial timestamps. End-to-end policy inference latency, GPU utilization, and the achieved policy-query period were not measured separately, so the nominal 5 Hz action cadence and

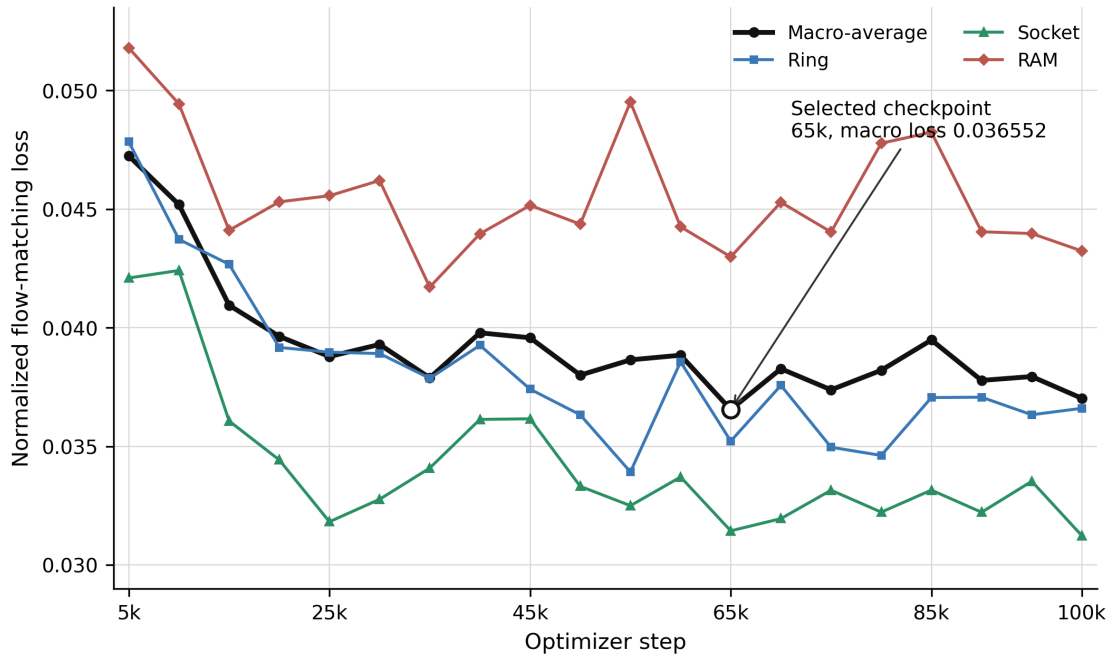


Illustration 5–1 Task-specific and unweighted macro normalized flow-matching loss across the 100,000-step training run. The 65,000-step checkpoint has the minimum recorded macro loss and was selected before physical testing.

approximately two-second executed chunk remain interface settings rather than measured latency results.

5.4 Physical trial protocol and success definition

For each trial, the prepared scene corresponded to one of the three fixed language instructions in Table 4–11. The same 65,000-step checkpoint was used for every task. Task identity was provided through the language instruction; separate task-specific checkpoints were not loaded.

The objects were reset before every trial and their initial positions were randomly perturbed around the nominal task arrangement. The permitted placement variation was approximately 5 cm for the ring and RAM scenes and approximately 3 cm for the socket scene. For each task, the 25 trials were conducted consecutively. Once autonomous execution started, no manual intervention or corrective assistance was permitted during a counted trial.

A trial was counted as an end-to-end success only when its complete terminal condition

was achieved:

- for the ring task, the can was repositioned and the released ring remained over the can;
- for the socket task, the socket box was repositioned and the two-pin plug remained inserted in the two-hole socket; and
- for the RAM task, the memory module was fully removed from the motherboard and remained inside the yellow tray.

Failures were categorized by the first task stage that prevented the terminal condition. Partial completion was retained for failure analysis but was not counted as end-to-end success.

The counted-trial record preserves the task, language instruction, checkpoint, initial reset confirmation, autonomous start, terminal outcome, first blocking stage, intervention status, and stop reason. These fields prevent an assisted or partially completed trajectory from being reclassified as a success. The task trials were conducted after checkpoint selection; their outcomes were not used to choose among the 20 offline checkpoints.

The ES1, ES3, ES5, and ES6 tests use their own 25-experiment records. They are not added to the 75 ES2 denominator. Similarly, a recovery attempt recorded for ES6 is a response to a detected failure and is not counted as a new ES2 task success. This accounting prevents the same physical event from inflating more than one success rate.

The randomized reset evaluates tolerance to moderate initial-position variation rather than exact repetition of one fixed scene. However, the displacement limits are approximate and the individual trial coordinates were not logged. The results are therefore reported as descriptive success rates over the bounded reset procedure rather than as estimates over a precisely sampled statistical distribution.

5.5 Overall physical results

For task k , the end-to-end success rate is

$$R_k = \frac{N_k^{\text{success}}}{N_k^{\text{trial}}} \times 100\%. \quad (5-2)$$

Table 5–4 reports the resulting task-level values.

Because every task contains the same number of physical trials, the micro-average over all 75 trials and the unweighted macro-average of the three task success rates are both

$$\frac{59}{75} \times 100\% = \frac{84\% + 52\% + 100\%}{3} \approx 78.7\%.$$

Table 5–4 End-to-end physical success rates using the 65,000-step unified checkpoint

Task	Successful	Failed	Total	Success rate
Ring over can	21	4	25	84%
Plug into socket	13	12	25	52%
Memory-module removal	25	0	25	100%
All physical trials	59	16	75	78.7%

This aggregate value is higher than the 70% task-execution target in Table 2–2. However, the aggregate conceals a large task difference. The RAM and ring tasks individually exceed 70%, whereas the socket task reaches only 52%. The final system therefore does not satisfy the task-execution threshold uniformly across all three tasks.

5.5.1 Descriptive uncertainty of the observed rates

The raw count is the primary result, but a confidence interval helps show how much uncertainty remains with 25 task trials. For a binomial count x of n trials, the two-sided 95% Wilson interval is

$$\frac{\hat{p} + \frac{z^2}{2n} \pm z \sqrt{\frac{\hat{p}(1-\hat{p})}{n} + \frac{z^2}{4n^2}}}{1 + \frac{z^2}{n}}, \quad z = 1.96. \quad (5-3)$$

Wilson intervals are used because they remain well behaved at boundary results such as 25 successes in 25 trials.

Table 5–5 Observed task-success rates and Wilson 95% confidence intervals

Evaluation unit	Count	Observed rate	Wilson 95% interval
Ring over can	21/25	84.0%	65.3–93.6%
Plug into socket	13/25	52.0%	33.5–70.0%
Memory-module removal	25/25	100.0%	86.7–100.0%
All ES2 task trials	59/75	78.7%	68.1–86.4%

The point estimate is the pre-defined project acceptance statistic, so the overall 78.7% result passes ES2. The interval answers a different question: with 75 trials, the data still permit an underlying success probability below 70%. The final report therefore states that the observed project result meets the target; it does not claim statistical proof that a broader

population rate exceeds 70%. The same caution applies to the 100% RAM observation. Its lower Wilson bound is 86.7%, so 25 successful trials do not imply a zero failure probability.

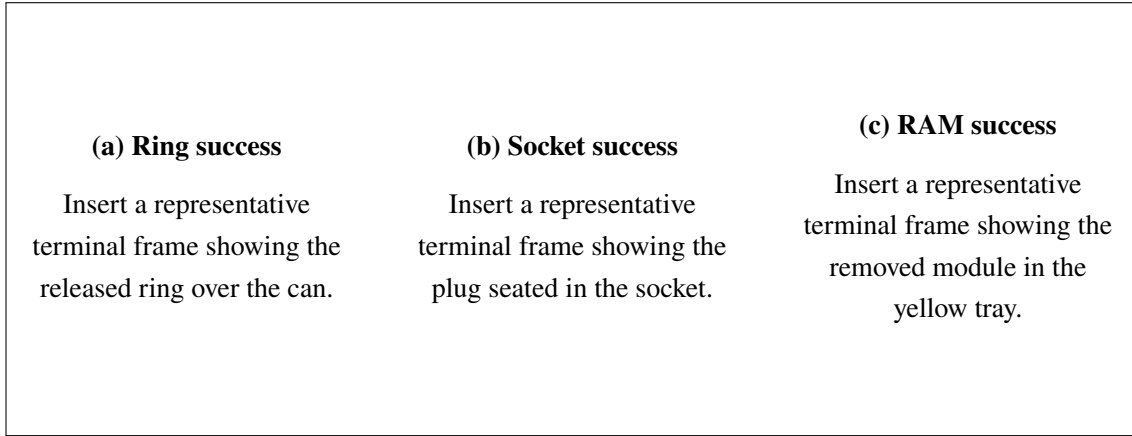


Illustration 5–2 Representative successful physical executions of the three final tasks. The three panels should use frames from counted trials and should show the complete terminal condition rather than an intermediate grasp or approach.

5.6 Task-specific results and failure analysis

5.6.1 Memory-module removal and tray placement

The RAM task succeeded in all 25 counted trials, giving an observed success rate of 100%. In the successful sequence, the left arm pulled and stabilized the motherboard while the right arm removed the memory module and transferred it to the yellow tray. No end-to-end failure was recorded in the 25-trial set.

This result shows that the selected demonstration data, bilateral role allocation, camera observations, compliant grippers, and unified checkpoint were sufficient for the evaluated motherboard configuration. The 25-out-of-25 result should nevertheless be interpreted as performance within the tested setup. A finite trial set with no observed failure does not establish a zero failure probability, and the current evaluation does not quantify sensitivity to motherboard pose, memory-module type, connector retention force, lighting, or camera displacement.

5.6.2 Ring placement over the can

The ring task succeeded in 21 of 25 trials, corresponding to 84%. Four trials failed. Table 5–6 separates the observed failure modes.

Table 5–6 Observed failure modes in the ring task

Failure mode	Count	Share of ring failures
Ring not successfully placed over the can	1	25%
Right gripper deviated from the handle and missed the grasp	2	50%
Handle grasp failed because the ring base obstructed the approach	1	25%
Total	4	100%

Three of the four failures occurred before secure transport because the right gripper did not successfully acquire the ring handle. The remaining failure occurred at the ring-over-can placement stage. The observed distribution therefore identifies handle acquisition as the dominant bottleneck in this task, rather than the initial can-pushing operation.

In two failures, the right gripper approached with an offset from the handle and therefore did not establish a grasp. The obstruction failure shows that the ring base can remove the approach clearance required by the custom fingers. These observations motivate additional demonstrations around displaced handle configurations, inspection of wrist-camera visibility near the ring base, and evaluation of a more reliable pre-grasp alignment. They also provide a concrete basis for evaluating whether a revised finger profile or a more vertical pre-grasp approach would improve robustness; these changes are proposed from the observed failures and were not evaluated in the present trial set.

5.6.3 Two-pin plug grasping and two-hole socket insertion

The socket task succeeded in 13 of 25 trials, corresponding to 52%. Twelve trials failed. Table 5–7 reports the two observed failure categories.

The dominant failure is constrained insertion rather than plug acquisition. Ten of the twelve failures reached the stage at which the two-pin plug had been grasped but did not enter the two-hole socket accurately enough to satisfy the terminal condition. This is consistent

Table 5–7 Observed failure modes in the socket task

Failure mode	Count	Share of socket failures
Left gripper did not acquire or retain the plug	2	16.7%
Two-pin plug grasped, but alignment with the two-hole socket was inaccurate	10	83.3%
Total	12	100%

with the tolerance analysis in Section 4.2.6: the insertion requires both lateral alignment of the two pins with their corresponding holes and sufficiently small angular error through the insertion motion.

The result does not by itself identify a single root cause. Possible contributors include residual socket-box pose variation after the right-arm push, plug pose variation inside the compliant left gripper, limited visual information at the contact point, and the temporal interval over which ten predicted actions are executed before replanning. These are engineering hypotheses inferred from the failure stage, not measured causal conclusions. Confirming them would require trial-level video review, socket and plug geometry measurements, and preferably logging of the wrist view and commanded joint targets immediately before contact.

5.7 Planning, detection, pose-error, failure-detection, and recovery results

5.7.1 Rate-based engineering specifications

ES1, ES3, ES5, and ES6 are each evaluated over 25 experiments. Their point estimates and Wilson intervals are shown in Table 5–8. The threshold decision uses the point estimate, while the interval exposes the remaining sampling uncertainty.

For ES1, 23 of 25 initialized task cases formed an executable plan with the loaded unified checkpoint and the selected instruction. The 92% point estimate exceeds the greater-than-90% target. This result shows that the prepared scene and task interface usually entered a valid executable state. It does not establish general high-level planning across unseen tasks because the test uses the three documented instructions and workcell configurations.

For ES3, the required component was correctly detected in 24 of 25 evaluated observa-

Table 5–8 Rate-based ES1, ES3, ES5, and ES6 results

ES	Measured outcome	Target	Count	Observed rate	Wilson 95% in- ter- val
ES1	Executable-plan formation	> 90%	23/25	92.0%	75.0– 97.8%
ES3	Required-component detection	> 90%	24/25	96.0%	80.5– 99.3%
ES5	Failure detection before further unsafe action	> 90%	23/25	92.0%	75.0– 97.8%
ES6	Recovery to a valid task state	> 70%	23/25	92.0%	75.0– 97.8%

tions. The 96% point estimate exceeds the greater-than-90% target. The result applies to the handled ring, plug and socket, motherboard, memory module, can, and tray under the final camera arrangement. It should not be reported as a category-level detector benchmark or as evidence for arbitrary components, because component identity and appearance were bounded by the final task set.

For ES5, 23 of 25 evaluated failures were identified before an additional unsafe action was issued. The 92% point estimate exceeds the greater-than-90% target. The detection path operates around the learned policy: camera freshness, packet validity, finite action values, bounds, communication state, robot protective stops, and task-level failure conditions contribute to the decision. Passing ES5 under this test does not constitute functional-safety certification or quantify false alarms under unrestricted operation.

For ES6, the defined recovery procedure restored a valid task state in 23 of 25 recovery opportunities, giving 92% and exceeding the greater-than-70% target. Recovery outcomes are not added to the original ES2 numerator. This separation prevents a failed initial task attempt followed by recovery from being reported as an uninterrupted end-to-end success.

The 95% intervals for ES1, ES5, and ES6 are identical because their counts are identical. For ES1, ES3, and ES5, the lower interval bounds fall below the design threshold even though the point estimates pass. The evidence therefore supports the project acceptance decisions

for these 25-experiment sets but is not a statistical guarantee that the underlying rates are greater than 90%.

5.7.2 ES4 pose-error result and reporting rule

ES4 requires component position error below 20 mm. The final record reports that the measured position errors were below 20 mm. Successful executions were not assigned a synthetic error of zero; they were excluded from the recorded-error subset. This rule avoids artificially reducing an average by treating task success as perfect localization.

The available summary does not include the individual error values or the number of non-success cases for which a metric error was recorded. The result is therefore reported as an operational threshold outcome:

$$e_{\text{recorded}} < 20 \text{ mm.}$$

No mean, median, maximum, standard deviation, confidence interval, or 25-sample error distribution is calculated. Under the supplied measurement rule, ES4 passes its project limit; a stronger pose-estimation claim would require the trial-level reference and estimated coordinates, the measurement method, and the complete inclusion set.

5.8 Cross-task interpretation

The results demonstrate that one 65,000-step $\pi_{0.5}$ checkpoint can produce successful physical executions for all three language-selected tasks without loading separate policies. Performance is nevertheless task dependent. The RAM task was fully successful in the observed trial set, the ring task was largely successful but sensitive to handle acquisition, and the socket task remained limited by precision insertion.

The failure distribution also separates gross bimanual sequencing from local contact precision. No failure category in the supplied record was assigned to choosing the wrong arm role or loading the wrong task-specific checkpoint. The observed failures instead occur at local grasp or placement/insertion stages. This supports the feasibility of the unified bilateral action interface while showing that unified task representation does not remove the need for task-specific contact robustness.

The current three scenes are visually distinct and each scene is paired with its expected

instruction. Consequently, these trials validate deployment of a language-conditioned unified checkpoint, but they do not isolate how strongly the learned behavior depends on the language input. A separate instruction-sensitivity experiment using paraphrased, exchanged, or deliberately mismatched instructions would be required to distinguish language grounding from task recognition based primarily on scene appearance.

5.8.1 Requirements-level interpretation

At the project level, ES1 through ES6 all pass their stated point-estimate or operational threshold. The strongest evidence is not any one percentage but the chain across specifications: the configured scene and instruction generally form an executable plan; required components are detected; the recorded pose error remains below the operational limit; the same unified checkpoint completes all three task types; and most tested failure and recovery events are handled successfully.

The compliance statement does not mean that every task is equally mature. ES2 is defined as an overall task-execution rate and reaches 78.7%, but socket insertion reaches only 52%. A sponsor deciding whether to use the prototype for the socket operation should therefore treat the socket-specific value, not the overall average, as the relevant design signal. The mixed result is engineering evidence for targeted redesign rather than a reason to conceal the aggregate pass or the task-level failure.

The result also separates system integration from policy comparison. ACT remains the DR3 deployable baseline, but no matched ACT trial set is available under the final three-task protocol. The present evaluation shows that the final $\pi_{0.5}$ route works within the scoped work-cell and meets the project acceptance rules. It does not establish that $\pi_{0.5}$ is more accurate, safer, faster, or more data-efficient than ACT.

5.9 Validation limitations

Several limitations constrain interpretation of the reported results.

1. Although the objects were randomly perturbed within approximately 5 cm for the ring and RAM tasks and 3 cm for the socket task, the individual initial coordinates and exact sampling distribution were not logged. The experiment demonstrates tolerance to bounded resets, not a quantified spatial-generalization distribution.

2. The approximately one-minute completion time was not recorded as a per-trial timing series, and end-to-end inference latency was not measured separately. The nominal 5 Hz cadence and ten-action segment are configured interfaces rather than measured timing performance.
3. The evaluation contains one selected checkpoint and no ACT, single-task, or alternative-checkpoint physical baseline. It demonstrates final-system performance but not the superiority of $\pi_{0.5}$ over the DR3 baseline.
4. The visually distinct scenes were evaluated with their corresponding instructions. The trials therefore validate an instruction-conditioned common interface but do not independently measure sensitivity to instruction wording or separate language grounding from scene recognition.
5. Each rate-based ES1, ES3, ES5, and ES6 result contains 25 observations. The point estimates meet the project thresholds, but their Wilson intervals remain wide. Larger pre-registered test sets would be needed for precise population-level rate claims.
6. ES4 is available only as a recorded-error threshold result. The individual errors and inclusion count were not supplied, so the result cannot support a continuous error distribution or a statistical pose analysis.
7. The exact training source was not archived as one clean Git commit, the environment file is incomplete, and the base-weight conversion procedure is not retained. The deployed checkpoint is identifiable and portable, but exact retraining remains only partially reproducible.

These limitations do not change the counted physical outcomes or the project point-estimate judgments, but they define the boundary of the supported claims. The strongest supported conclusion is that the 65,000-step unified checkpoint executed all three scoped tasks on the physical dual-arm setup with task-dependent reliability and that the six project specifications passed their stated final decision rules. Broader claims about generalization, timing, language sensitivity, comparative policy performance, statistical population rates, exact retraining, or production readiness require the additional measurements identified above.

5.10 Validation summary

The 65,000-step checkpoint, selected by the minimum task-balanced macro flow-matching loss of 0.036552, was deployed on an NVIDIA GeForce RTX 5070 Ti and evaluated in 75 physical trials. Objects were reset and randomly perturbed before every trial, the 25 trials for each task were conducted consecutively without manual intervention, and a complete execution took approximately one minute. The memory-module task achieved 25/25 successes, the ring task achieved 21/25, and the socket task achieved 13/25. The combined result was 59/75, or 78.7%.

The additional engineering-specification tests produced ES1 planning success of 23/25 (92%), ES3 component detection of 24/25 (96%), ES5 failure detection of 23/25 (92%), and ES6 recovery of 23/25 (92%). The recorded ES4 error remained below 20 mm under the supplied rule that successful executions were not assigned a zero error. All project-level point estimates or operational measurements meet their stated thresholds, but their small-sample and protocol boundaries remain explicit.

Failure analysis shows that ring failures were dominated by unsuccessful handle acquisition, while socket failures were dominated by inaccurate alignment and insertion into the two-hole socket. The RAM and ring tasks exceeded the 70% task-execution target, but the socket task did not. The validation therefore confirms successful unified multi-task deployment and project-level specification compliance under the evaluated conditions, while identifying precision plug insertion as the principal remaining performance bottleneck.

Chapter 6 Discussion and conclusions

6.1 Discussion

This project developed a bimanual industrial manipulation system for representative 3C assembly operations. The final system integrates a dual-Franka Research 3 platform, GELLO-based demonstration collection, synchronized visual and proprioceptive data recording, task-compatible end effectors and fixtures, and a unified $\pi_{0.5}$ VLA policy deployment workflow. The design process remains traceable from customer requirements to engineering specifications, concept generation, concept selection, implementation, and validation.

The principal validation result is that one 65,000-step checkpoint executed all three language-selected tasks on the physical dual-arm workcell without loading separate task-specific policies. Across 75 counted trials, the system completed 59 trials successfully, giving an observed overall success rate of 78.7%. The memory-module task succeeded in 25 of 25 trials (100%), the ring task succeeded in 21 of 25 trials (84%), and the plug-insertion task succeeded in 13 of 25 trials (52%). Separate 25-experiment tests produced ES1 planning success of 92%, ES3 component detection of 96%, ES5 failure detection of 92%, and ES6 recovery of 92%; the recorded ES4 error remained below 20 mm under its stated measurement rule. All project-level point estimates or operational measurements met their thresholds, but performance was not uniform: the plug-insertion task did not individually meet the 70% ES2 target.

The strongest aspect of the design is its system-level integration. The two FR3 arms, bilateral GELLO demonstration interface, three-camera observation, 16-dimensional state and action representation, LeRobot data pipeline, and unified $\pi_{0.5}$ checkpoint operated together in real-robot trials. No reported failure was attributed to selecting the wrong arm role or loading an incorrect task-specific checkpoint. This supports the feasibility of using one language-conditioned bilateral policy interface for multiple tasks. The 100% observed success rate in the memory-module task also indicates that the demonstrated division of work—left-arm stabilization followed by right-arm extraction and placement—was well represented in the data for the evaluated motherboard configuration.

The ring results show that the system can perform a multi-stage reposition–grasp–place

sequence reliably under moderate initial-position variation, but they also expose sensitivity at the grasp interface. Three of the four ring failures occurred before secure transport: two were caused by the right gripper approaching away from the handle, and one occurred when the ring base obstructed the handle approach. Only one failure occurred during placement over the can. The dominant limitation is therefore handle acquisition rather than the initial can-pushing motion or the overall bilateral sequence. Improved wrist-camera visibility, additional demonstrations near displaced handle configurations, and a more repeatable pre-grasp approach would directly address the observed failure pattern.

The plug-insertion task is the least successful and the clearest remaining engineering bottleneck. Two of its twelve failures occurred because the left gripper did not acquire or retain the plug, while ten failures (83.3% of socket failures) occurred after grasping because the two pins were not aligned accurately enough with the two socket holes. The system can therefore complete the gross bimanual sequence of repositioning the socket box and acquiring the plug, but it lacks sufficient local contact precision for reliable insertion. Likely contributors include residual socket-box pose variation, plug-pose variation within the compliant gripper, limited visual information at the contact point, and the approximately two-second interval over which ten predicted actions are executed before replanning. These explanations are consistent with the observed failure stage but remain hypotheses because contact force, insertion depth, trial-level pose, and pre-contact action traces were not measured.

Several limitations restrict the conclusions that can be drawn from the results. The work-cell, task objects, fixtures, and lighting were controlled, and the exact randomized initial coordinates were not logged. Each visually distinct scene was paired with its expected instruction, so the study demonstrates deployment of a language-conditioned unified checkpoint but does not isolate whether task selection depends primarily on language or scene appearance. The evaluation also contains no ACT, task-specific, or alternative-checkpoint physical baseline and therefore does not establish that $\pi_{0.5}$ is superior to the earlier controller. The rate-based ES tests contain only 25 observations each, the ES4 raw error series is unavailable, and the exact training source was not archived as one clean commit; these facts limit statistical and reproducibility claims without changing the recorded outcomes.

If additional development time were available, the first priority would be to improve plug alignment and insertion through a more constrained fixture or lead-in geometry, a clearer

close-range view, shorter action execution between observations, and force- or torque-informed contact handling. The ring task would benefit from targeted demonstrations and pre-grasp alignment around difficult handle poses. A subsequent evaluation should record initial object coordinates and timing, test instruction paraphrases and mismatches, include comparison baselines, and evaluate explicit failure detection and bounded recovery. These changes would preserve the demonstrated unified multi-task architecture while addressing the local precision and validation weaknesses revealed by the final trials.

6.2 Conclusions

The thesis presents an end-to-end engineering process for VLA-oriented bimanual industrial manipulation. Starting from customer requirements, the project defined measurable engineering specifications, generated modular solution concepts, selected a dual-FR3 and GELLO-based architecture, and implemented a data-driven manipulation pipeline using synchronized sensing and one jointly trained $\pi_{0.5}$ checkpoint. The final controller receives three RGB observations, a 16-dimensional bilateral robot state, and a language instruction, and generates coordinated actions for both robot arms and grippers.

The completed system demonstrates language-conditioned multi-task bimanual control on the physical dual-FR3 workcell. The same unified checkpoint was evaluated in 25 trials for each of the three tasks, giving 75 physical trials in total. The ring-over-can task achieved an 84% success rate, the plug-insertion task achieved 52%, and the memory-module task achieved 100%. Across all trials, the observed overall success rate was 78.7%.

The overall result exceeded the 70% task-execution target. The separate ES1, ES3, ES5, and ES6 point estimates were 92%, 96%, 92%, and 92%, respectively, and the recorded ES4 error remained below 20 mm under its stated rule. All six project-level acceptance rules were therefore met under the evaluated conditions. However, performance was not uniform across tasks: the ring and memory-module tasks exceeded the ES2 target, whereas the plug-insertion task did not. The final result establishes a working and traceable proof of concept while identifying contact-rich plug alignment and insertion as the principal remaining performance limitation.

Chapter 7 Future Work

The completed prototype establishes a functional basis for language-conditioned bimanual manipulation, but it remains a laboratory proof of concept rather than a production-ready assembly system. Future development should therefore proceed at two levels. At the system level, the work should improve robustness, generalization, autonomous failure handling, and deployment readiness. At the detailed engineering level, it should close the measurement and documentation gaps identified during implementation and validation. The recommendations below are intended to provide the project sponsor and future student teams with a practical continuation plan.

7.1 Improve contact-rich task performance

The first priority is the plug-insertion task. Its observed success rate was 52%, compared with 84% for the ring task and 100% for the memory-module task. Future work should first separate failures occurring during socket-box positioning, plug grasping, alignment, and final insertion. Each stage should be given an independently observable terminal condition so that additional demonstrations can be targeted at the actual bottleneck rather than collected uniformly across the entire task.

The insertion stage would benefit from a more constrained mechanical and sensing design. Recommended changes include adding a passive lead-in chamfer or alignment fixture, improving plug-grasp repeatability, reducing socket-box motion during insertion, and recording a close-range view that clearly exposes the plug pins and socket openings. If the available robot interface permits it, force or joint-torque feedback should be used to detect premature contact and to execute a compliant search or guarded insertion motion. The redesigned system should then be evaluated using the same end-to-end success definition and at least the same number of trials as the present study so that the result can be compared directly with the 52% baseline.

7.2 Develop autonomous failure detection and recovery

The current system can stop safely when communication, state, action, or workspace checks fail, but it does not yet perform task-level recovery after a manipulation error. A future controller should distinguish among recoverable failures, unrecoverable failures, and safety-critical conditions. Recoverable events may include a missed grasp, an object displaced within the reachable workspace, an incomplete insertion, or a ring that remains graspable after a failed placement. Protective stops, unexpected human entry, invalid sensor data, and objects leaving the verified workspace should continue to require operator intervention.

A practical recovery architecture can combine deterministic checks with learned visual assessment. After each important task stage, the controller should evaluate whether the expected state has been reached. If not, it should select a bounded recovery routine such as reopening the gripper, retreating to a safe pose, reacquiring the object, or repeating the final alignment. Every recovery attempt should be logged separately from the initial attempt. Future validation should report the failure-detection rate, false-alarm rate, recovery-attempt rate, and recovery success rate, including whether the current engineering target of greater than 70% recovery success is achieved.

7.3 Expand data coverage and model evaluation

The present validation covers controlled scenes with moderate object-position perturbations. Future datasets should systematically vary object pose, lighting, camera displacement, background clutter, component type, and fixture tolerance. Initial conditions should be sampled from defined distributions and recorded numerically for every episode and physical trial. This would allow performance to be reported as a function of perturbation magnitude rather than as a single success percentage over an approximately defined reset region.

Data collection should emphasize failure-prone and boundary conditions while preserving task balance. Failed demonstrations, recovery demonstrations, and successful corrections should be retained with explicit stage and outcome labels. Dataset versions should record episode counts, frame counts, camera calibration, operator identity, task instruction, object configuration, and the software commit used for collection. A held-out test set should remain unchanged across model revisions so that improvements are not inferred from repeat-

edly tuned validation data.

The unified $\pi_{0.5}$ checkpoint should also be compared with meaningful baselines. At minimum, future experiments should include the DR3 ACT implementation, task-specific checkpoints, and alternative checkpoints from the same $\pi_{0.5}$ training run. Ablation studies should evaluate the contribution of language instructions, individual camera views, bilateral state input, equal-probability task sampling, and action-chunk execution length. These comparisons are necessary to determine which parts of the final architecture produce the observed performance rather than merely demonstrating that the integrated system can operate.

7.4 Improve language conditioning and task generalization

The current evaluation uses one fixed instruction for each visually distinct task. Consequently, it does not independently establish robustness to instruction wording or the ability to disambiguate tasks from language alone. Future testing should introduce paraphrases, re-ordered but equivalent instructions, irrelevant wording, and deliberately conflicting instructions. Visual scenes should also be designed so that the same objects can support more than one valid operation, making language information necessary for selecting the intended behavior.

Longer-term work should investigate compositional task execution. Instead of learning each complete sequence only as a fixed demonstration pattern, the system could represent reusable skills such as locating, grasping, stabilizing, transporting, aligning, inserting, and releasing. A high-level instruction could then select and order these skills while the action model performs continuous control. Any such extension should preserve the deterministic safety boundary and should not permit language-model output to bypass robot-side joint, workspace, communication, or emergency-stop constraints.

7.5 Close mechanical, sensing, and timing documentation gaps

Several quantities required for production-level reproducibility were not measured in the present project. A future team should create a dimensioned workcell record containing the two FR3 base poses, fixture coordinates, camera positions and orientations, usable shared workspace, and verified inter-arm clearance. Camera intrinsic and extrinsic calibra-

tion should be stored with each dataset version, and calibration drift should be checked after mount adjustment or accidental contact.

The custom TPU fingers should be documented with slicer version, print orientation, layer height, wall count, infill, nozzle and bed temperatures, print mass, print time, and fastening torque. Simple force–deflection and repeated-cycle tests should quantify compliance, permanent deformation, and failure. The ring/can clearance, plug/socket clearance and insertion depth, and memory-module extraction clearance should be measured using the available ruler and caliper rather than inferred from images.

The deployment pipeline should record timestamps at image acquisition, packet assembly, policy-query start and completion, command publication, and robot-state feedback. These measurements would distinguish the configured 5 Hz action cadence from the achieved closed-loop rate and would identify whether image transport, model inference, network communication, or robot execution is the dominant source of delay.

7.6 Prepare for a more reproducible and deployable system

Future releases should package the software environment, configuration files, dataset metadata, calibration records, model checkpoint, and run procedure as one versioned deployment bundle. The Bill of Materials should be updated with supplier, catalog number, version, quantity, unit cost, and replacement value for both physical and virtual components. A reproduction checklist should allow a new team to reconstruct the workcell, validate device identities and coordinate conventions, perform a dry run, and reproduce the reported evaluation without relying on undocumented knowledge from the original team.

For operation beyond supervised laboratory trials, the workcell will also require a formal risk assessment. The assessment should define guarded operating zones, speed and force limits, inter-arm collision handling, fixture retention requirements, emergency-stop access, safe reset procedures, and criteria for rejecting a scene before autonomous execution. Production deployment should be considered only after the system demonstrates repeatable performance across multiple operators, days, object instances, and controlled environmental variations.

7.7 Recommended continuation sequence

To avoid expanding the project before its present weaknesses are resolved, future work should follow a staged sequence:

1. reproduce the current 75-trial result using the archived checkpoint, configuration, and task definitions;
2. complete the missing dimensional, calibration, fabrication, cost, and latency records;
3. improve the plug-insertion mechanics and collect targeted demonstrations until the task independently exceeds the 70% success target;
4. implement stage-level failure detection and bounded recovery, then evaluate recovery against the defined engineering specifications;
5. compare the unified $\pi_{0.5}$ model with ACT, task-specific models, and controlled ablations; and
6. expand language and scene variation only after the revised system remains stable under the original evaluation protocol.

This sequence preserves a measurable baseline while progressively addressing the main limitation revealed by the final validation. It also gives future teams a clear separation between documentation work, engineering redesign, learning-system experiments, and higher-level research extensions.

References

- [1] BROHAN A, et al. RT-2: Vision-Language-Action Models Transfer Web Knowledge to Robotic Control[C]//Conference on Robot Learning (CoRL). 2023. arXiv: 2307.15818.
- [2] KIM M J, et al. OpenVLA: An Open-Source Vision-Language-Action Model[J]. ArXiv preprint, 2024. arXiv: 2406.09246.
- [3] LIU S, et al. RDT-1B: A Diffusion Foundation Model for Bimanual Manipulation[J]. ArXiv preprint, 2024. arXiv: 2410.07864.
- [4] BLACK K, et al. π_0 : A Vision-Language-Action Flow Model for General Robot Control[J]. ArXiv preprint, 2024. arXiv: 2410.24164.
- [5] ZENG A, et al. Transporter Networks: Rearranging the Visual World for Robotic Manipulation[C]//Conference on Robot Learning (CoRL). 2020. arXiv: 2010.14406.
- [6] SHRIDHAR M, MANUELLI L, FOX D. CLIPort: What and Where Pathways for Robotic Manipulation[C]//Conference on Robot Learning (CoRL). 2021. arXiv: 2109.12098.
- [7] SHRIDHAR M, MANUELLI L, FOX D. Perceiver-Actor: A Multi-Task Transformer for Robotic Manipulation[C]//Conference on Robot Learning (CoRL). 2022. arXiv: 2209.05451.
- [8] WALKER H, et al. BridgeData V2: A Dataset for Robot Learning at Scale[C]//Conference on Robot Learning (CoRL). 2023. arXiv: 2308.12952.
- [9] Open X-Embodiment Collaboration, et al. Open X-Embodiment: Robotic Learning Datasets and RT-X Models[C]//Conference on Robot Learning (CoRL). 2023. arXiv: 2310.08864.
- [10] KHAZATSKY A, et al. DROID: A Large-Scale In-the-Wild Robot Manipulation Dataset[C]//Robotics: Science and Systems (RSS). 2024. arXiv: 2403.12945.
- [11] LIU B, et al. LIBERO: Benchmarking Knowledge Transfer for Lifelong Robot Learning[C]//Advances in Neural Information Processing Systems (NeurIPS). 2023. arXiv: 2306.03310.
- [12] BLACK K, et al. $\pi_{0.5}$: A Vision-Language-Action Model with Open-World Generalization[J]. ArXiv preprint, 2025. arXiv: 2504.16054.
- [13] Physical Intelligence. OpenPI: Open-Source Models and Packages for Robotics[EB/OL]. [2026-07-27]. <https://github.com/Physical-Intelligence/openpi>.
- [14] Hugging Face. LeRobot: Making AI for Robotics More Accessible with End-to-End Learning[EB/OL]. [2026-07-27]. <https://github.com/huggingface/lerobot>.
- [15] Hugging Face. $\pi_{0.5}$ (Pi05) Policy in LeRobot[EB/OL]. [2026-07-27]. <https://github.com/huggingface/lerobot/blob/main/docs/source/pi05.mdx>.
- [16] MANDLEKAR A, et al. What Matters in Learning from Offline Human Demonstrations for Robot Manipulation[C]//Conference on Robot Learning (CoRL). 2021. arXiv: 2108.03298.
- [17] MANDLEKAR A, et al. MimicGen: A Data Generation System for Scalable Robot Learning Using Human Demonstrations[C]//Conference on Robot Learning (CoRL). 2023. arXiv: 2310.17596.
- [18] WU P, SHENTU Y, YIZ, et al. GELLO: A General, Low-Cost, and Intuitive Teleoperation Framework for Robot Manipulators[C]//IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS). 2024. arXiv: 2309.13037.
- [19] CHI C, et al. Universal Manipulation Interface: In-The-Wild Robot Teaching Without In-The-Wild Robots[C]//Robotics: Science and Systems (RSS). 2024. arXiv: 2402.10329.
- [20] ZHAO T Z, et al. Learning Fine-Grained Bimanual Manipulation with Low-Cost Hardware[C]//Robotics: Science and Systems (RSS). 2023.

- [21] CHI C, et al. Diffusion Policy: Visuomotor Policy Learning via Action Diffusion[C]//Robotics: Science and Systems (RSS). 2023. DOI: 10.15607/RSS.2023.XIX.026.
- [22] ZE Y, et al. 3D Diffusion Policy: Generalizable Visuomotor Policy Learning via Simple 3D Representations[C]//Robotics: Science and Systems (RSS). 2024. arXiv: 2403.03954.
- [23] NAIR S, RAJESWARAN A, KUMAR V, et al. R3M: A Universal Visual Representation for Robot Manipulation[C]//Conference on Robot Learning (CoRL). 2022. arXiv: 2203.12601.
- [24] GOYAL A, et al. RVT: Robotic View Transformer for 3D Object Manipulation[C]//Conference on Robot Learning (CoRL). 2023. arXiv: 2306.14896.
- [25] Franka Robotics. Franka Research 3 Datasheet[EB/OL]. [2026-07-27]. https://download.franka.de/Franka-Research-3_Datasheet_v1.1_August2022.pdf.
- [26] Franka Robotics. Franka Research 3[EB/OL]. [2026-07-27]. <https://franka.de/franka-research-3>.
- [27] Franka Robotics. Franka Control Interface Documentation[EB/OL]. [2026-07-27]. <https://support.franka.de/docs/overview.html>.
- [28] RealSense. RealSense Depth Camera D435i: Technical Specifications[EB/OL]. [2026-07-27]. <https://realsenseai.com/products/depth-camera-d435i/>.
- [29] RealSense. RealSense Depth Camera D405: Technical Specifications[EB/OL]. [2026-07-27]. <https://realsenseai.com/products/stereo-depth-camera-d405/>.
- [30] PICO. PICO 4 Ultra[EB/OL]. [2026-07-27]. <https://www.picoxr.com/global/products/pico4-ultra>.
- [31] ROBOTIS. DYNAMIXEL XL330-M288-T: Specifications[EB/OL]. [2026-07-27]. <https://emanual.robotis.com/docs/en/dxl/x/xl330-m288/>.

Documentation of Generated Concepts Appendix 1

Purpose and design evolution

This appendix preserves the alternatives generated during the DR3 concept-development process and records the implementation-stage change from an ACT baseline to the LeRobot $\pi_{0.5}$ policy. The alternatives were retained because they expose the principal engineering trade-offs in platform selection, teleoperation, sensing, end-effector design, and learned control. The update to $\pi_{0.5}$ does not invalidate the earlier concept generation; it documents how implementation evidence and the requirement for language-conditioned multi-task control changed the final policy decision.

Table 1–1 Evolution from the DR3 baseline to the final implementation

Design aspect	DR3 baseline	Final implementation
Robot platform	Dual FR3 selected; single-arm evidence used for staged risk reduction	Dual FR3 physical execution
Policy	ACT in LeRobot; task-specific checkpoint selection	$\pi_{0.5}$ in LeRobot with language-conditioned task input
Task scope	Ring placement and plug insertion used as staged cases	Ring over can, plug insertion, and memory-module removal
Language	Future extension	Runtime task instruction supplied to the VLA policy
Control scope	Vision–action imitation-learning baseline	Vision–language–action bimanual control

Module alternatives retained after concept generation

C1: Robot-platform alternatives

The selected platform is the dual-FR3 workcell because it provides a direct controller path within the available laboratory ecosystem and supports coordinated bimanual manipulation. A dual Flexiv platform remains attractive for force-aware contact tasks, but it would require a different controller, teleoperation interface, and data-integration path. An ALOHA-

style platform offers a lower-cost bimanual research route, but its mechanical capability and software ecosystem are less directly aligned with the existing FR3 workcell. The selected platform accepts higher integration effort in exchange for continuity with the available hardware and a direct route to the final dual-arm system.

C2: Teleoperation alternatives

GELLO was selected because its leader-arm structure provides an intuitive relationship between operator motion and the robot state/action trajectories recorded for learning^[18]. PICO 4 Ultra offers an XR-controller and hand-tracking route, but would add motion-mapping and calibration work^[30]. The Universal Manipulation Interface (UMI) provides a portable hand-held demonstration approach^[19]; it remains a useful research comparator but is less directly coupled to the fixed dual-FR3 joint-state pipeline.

C3: Data-collection alternatives

The selected concept uses synchronized multi-view RGB observations together with the state and action channels of both arms and grippers. The global view provides workspace and fixture context, while wrist or side views preserve information around grasping and contact regions. A reduced one-wrist-plus-global configuration lowers storage, calibration, and synchronization burden, whereas a proprioception-only configuration removes visual synchronization entirely. Both alternatives reduce the information available for language-grounded object selection, occlusion handling, and contact-region observation.

C4: End-effector alternatives

The standard Franka parallel gripper provides a direct robot interface and repeatable parallel-jaw motion, but offers limited passive adaptation to object geometry. A rigid 3D-printed gripper can be tailored to one component and can provide accurate locating features. The compliant finger concept retained from DR3 offers greater tolerance to small geometric and pose variation and can reduce concentrated contact forces during grasping and insertion. Because the final system contains three materially different tasks, the deployed end-effector configuration must be documented per task rather than assuming that one finger geometry is optimal for the ring handle, plug body, and memory module.

C5: Policy alternatives

ACT was the deployable DR3 baseline because it was integrated with the existing LeRobot demonstration workflow and could predict temporally structured actions from images and robot state^[20]. Diffusion Policy remains a credible visuomotor baseline for contact-rich manipulation^[21]. Neither model natively provides the same language-conditioned task interface required by the final three-task system.

The final implementation therefore uses $\pi_{0.5}$ through LeRobot^[12,15]. Compared with the ACT baseline, the selected VLA policy accepts a natural-language task description in addition to visual and proprioceptive observations. This supports a common interface for the three demonstrated tasks and aligns the deployed controller with the original VLA objective. The selection is an implementation decision; superiority over ACT or Diffusion Policy should only be claimed if controlled comparative trials are reported.

Generated task concepts

The final demonstrations instantiate three task concepts from the common bimanual VLA architecture:

1. workspace preparation followed by ring grasping and placement over a can;
2. workspace preparation followed by grasping, alignment, and plug insertion;
3. motherboard repositioning followed by memory-module removal and placement in a tray.

These tasks were retained because they collectively exercise object repositioning, language-grounded object selection, bimanual coordination, grasping, transport, constrained insertion, component extraction, and terminal placement. Their exact language instructions and completion conditions are listed in Table 4–11.

Hardware Specifications and Parts Records Appendix 2

Purpose

This appendix collects manufacturer reference parameters, component-level construction details, engineering drawings, and the complete parts record. These items support procurement, maintenance, replication, and hardware verification, but they are separated from Chapter 4 so that the main engineering analysis remains focused on design suitability, workspace, contact tolerance, sensing coverage, learning interfaces, and system operation.

Franka Research 3 reference parameters

Chapter 4 retains the four FR3 characteristics that directly justify the selected workcell: seven degrees of freedom, a 3 kg nominal payload, an 855 mm maximum reach, and link-side torque sensing in all seven axes. Table 2–1 preserves the broader manufacturer parameter set used during DR3^[25-27].

Figure 2–1 records the arm appearance, joint-axis convention, and principal link dimensions. Figure 2–2 preserves the complete DR3 parameter graphic. The workspace envelope remains in Figure 4–2 because it is used directly in the main shared-workspace analysis.

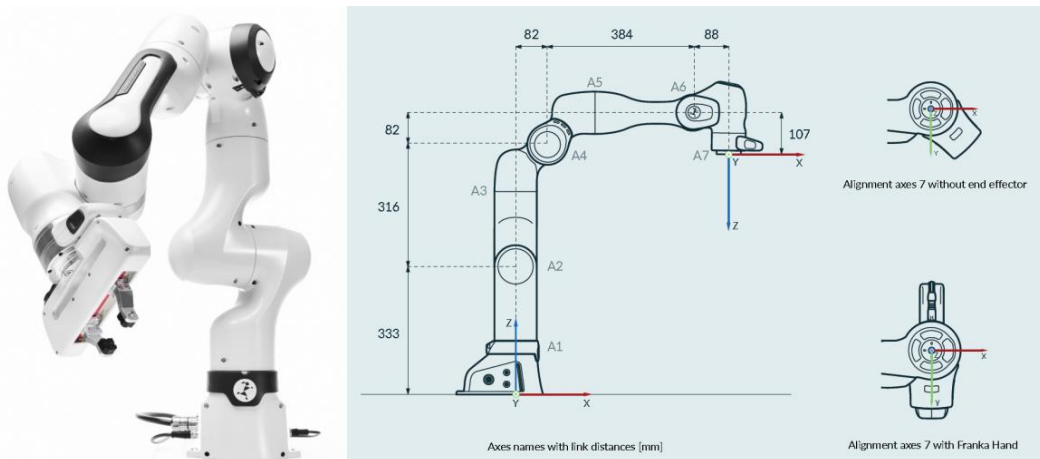


Illustration 2–1 FR3 arm, joint-axis convention, and principal kinematic dimensions retained as a hardware reference.

Table 2–1 FR3 manufacturer reference parameters used in the hardware record

Parameter	Reference value or interface
Degrees of freedom	7
Nominal payload	3 kg
Maximum reach	855 mm
Force/torque sensing	Link-side torque sensing in all seven axes
Joint position limits	A1 and A3: -166° to 166° ; A2: -105° to 105° ; A4: -176° to -7° ; A5: -165° to 165° ; A6: 25° to 265° ; A7: -175° to 175°
Mounting flange	DIN ISO 9409-1-A50
Installation position	Upright
Weight in the DR3 reference sheet	Approximately 17.8 kg; the value should be checked against the exact hardware and datasheet revision used in the laboratory
Protection rating	IP40
Ambient temperature	$+5^{\circ}\text{C}$ to $+45^{\circ}\text{C}$
Air humidity	20–80%, non-condensing
Primary interfaces	Ethernet (TCP/IP), control connector, end-effector connector, and safety-rated inputs
Project control boundary	High-level absolute targets are supplied through the project interface; low-level execution and manufacturer protection remain active

ARM			
Degrees of freedom	7	Interfaces	• ethernet (TCP/IP) for visual intuitive programming with Desk
Payload	3 kg		• safety-rated input for external enabling device
Maximum reach	855 mm		• 2 configurable safety-rated inputs for emergency stop devices, safeguards or other protective devices (OSSD devices via external OSSD converter connectable)
Force/Torque sensing	link-side torque sensor in all 7 axes		• hardware prepared for: 2x DI & 2x DO (24V, isolated, EN 61131-2 type 3 characteristics, 100 Hz sampling rate)
Joint position limits	A1, A3: -166/166 deg		• Control connector
	A2: -105/105 deg		• connector for end effector
	A4: -176/-7 deg	User Interfaces at the Arm's Pilot Grip	• integrated safety-rated guiding enabling switch
	A5: -165/165 deg		• guiding button
	A6: 25/265 deg		• guiding mode selector
	A7: -175/175 deg		
Mounting flange	DIN ISO 9409-1-A50	User Interfaces at the Arm's Pilot Disc	• status light
Installation position	upright		• Pilot mode selector
Weight	~ 17.8 kg		• arrow keys, teach, confirm, delete
Protection rating	IP40		
Ambient temperature ²	+5 °C to +45 °C		
Air humidity	20 – 80 % non-condensing		

Illustration 2–2 FR3 joint limits, mounting, environmental, and controller-interface parameters documented during DR3.

GELLO component-level construction

The final bilateral demonstration system uses one GELLO leader for each FR3 follower. Each leader supplies seven arm channels and one gripper-related channel. Table 2–2 lists the per-leader construction and the corresponding total for the dual-arm system.

Table 2–2 GELLO component-level construction

Component	Per leader	Dual total	Function
DYNAMIXEL XL330-M288-T	7	14	Measures the seven leader-arm joint channels
DYNAMIXEL XL330-M077-T	1	2	Provides the gripper-related input channel
ZNJ integrated control board	1	2	Reads and communicates the leader actuator states
Switched-mode power supply	1, 5 V 5 A	2	Supplies the leader electronics and actuator bus
FPX330-H101 structural kit	1 set	2 sets	Provides the modular leader structure
Printed links, brackets, and interfaces	1 set	2 sets	Completes the leader geometry, base, and Franka-compatible gripper input

The XL330 reference properties used during hardware selection are summarized in Table 2–3. They are component specifications rather than follower-robot requirements: the leader is used to measure and publish operator motion, not to reproduce the payload or torque capability of the FR3.

Table 2–3 DYNAMIXEL XL330 series reference properties

Property	Reference value
Input-voltage range	3.7–6 V
Recommended supply	5 V
Approximate dimensions	20 × 34 × 26 mm
Approximate mass	18 g per actuator
Supported control modes	Position, extended position, velocity, PWM, current, and current-based position modes
Selected variants	M288 for the seven arm joints and M077 for the gripper-related joint
Source	ROBOTIS XL330 documentation ^[31]

**Illustration 2–3 GELLO–Franka leader device retained as the component-level teleoperation reference.**

Compliant gripper drawing and fabrication reference

Each gripper assembly uses two identical TPU 64D fingers, giving four fingers in the dual-arm system. One finger has an overall bounding box of $123.36 \times 25.80 \times 40.00$ mm, corresponding to $L/T \approx 4.78$ and $L/H \approx 3.08$. It is screw mounted, with a reinforced attachment region and a tapered ribbed body.

Table 2–4 Compliant-finger fabrication reference

Item	Recorded design information
Material	TPU, Shore hardness 64D
Overall size	$123.36 \times 25.80 \times 40.00$ mm per finger
Quantity	Two fingers per gripper; four fingers for the dual-arm workcell
Attachment	Screw-mounted reinforced base
Contact geometry	Elongated tapered profile with repeated compliant ribs
Intended function	Passive accommodation of object-shape and pose variation during grasping and contact
Required inspection	Rib tearing, permanent bending, hole damage, paired-finger symmetry, screw retention, and opening clearance
Unreported quantities	Print orientation, infill, wall count, print mass and time, calibrated force–deflection curve, fatigue life, and contact load

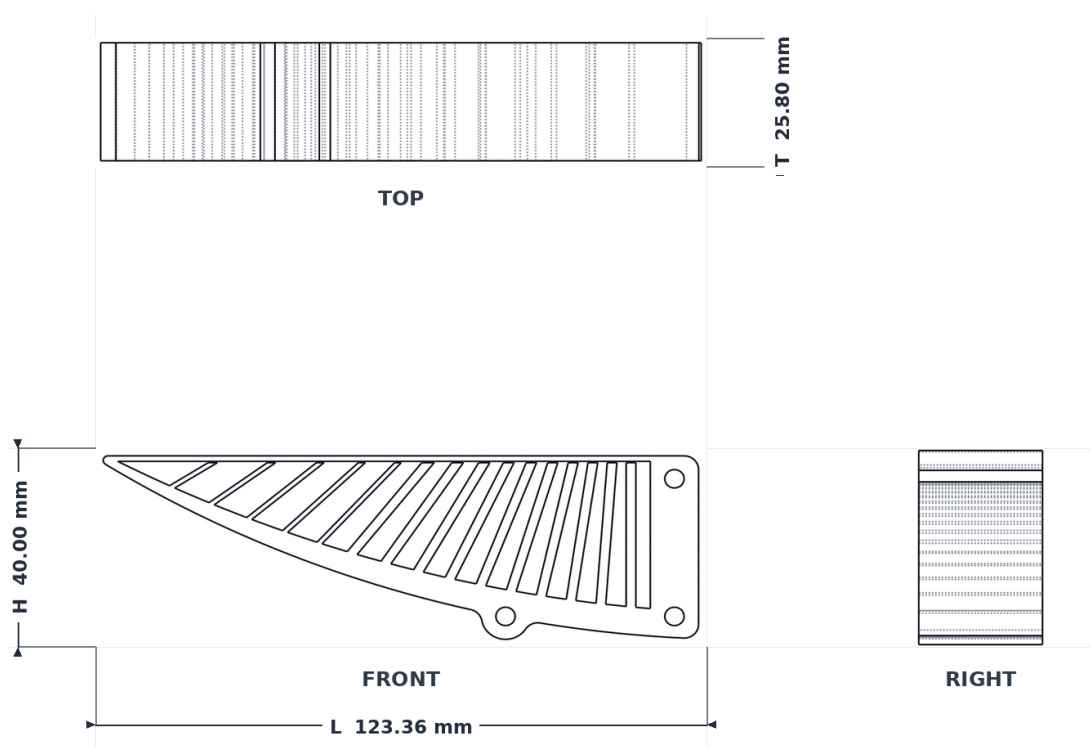


Illustration 2-4 Orthographic views of one TPU 64D compliant gripper finger and its overall dimensions.

Complete parts and components record

Table 2–5 preserves the complete system composition. Chapter 4 contains only a system-level summary so that component model numbers and procurement-oriented details do not interrupt the engineering argument.

Table 2–5 Complete parts and components record for the bimanual system

No.	Part or component	Specification	Qty.	Purpose
1	Franka Research 3 arm	7-DoF, 3 kg nominal payload, 855 mm maximum reach, joint torque sensing	2	Left and right follower manipulation
2	FR3 controller and robot interface	Manufacturer controller, network interface, low-level protection, FCI/ROS 2 integration	2 sets	State feedback and admissible joint-target execution
3	Custom gripper assembly	Project gripper with two screw-mounted compliant fingers	2	One end effector per arm
4	Compliant finger	TPU 64D, $123.36 \times 25.80 \times 40.00$ mm, tapered and ribbed	4	Object contact with passive geometric tolerance
5	Wrist camera	Intel RealSense D405, short-range RGB/depth device; RGB used by policy	2	Left- and right-wrist close-range observation
6	Overhead camera	Intel RealSense D435i; RGB used by policy	1	Global shared-workspace observation
7	GELLO leader assembly	7 arm channels plus 1 gripper channel	2	Bilateral human demonstration input
8	DYNAMIXEL XL330-M288-T	3.7–6 V actuator module; 5 V recommended	14	Seven arm joints per GELLO leader
9	DYNAMIXEL XL330-M077-T	3.7–6 V actuator module; 5 V recommended	2	Gripper-related channel on each GELLO leader
10	GELLO control board	ZNJ integrated control board	2	Reads and communicates leader joint positions
11	GELLO power supply	5 V, 5 A switched-mode supply	2	Leader-device electrical supply
12	GELLO structural kit	FPX330-H101 with project 3D-printed links and brackets	2 sets	Leader mechanism and mounting

No.	Part or component	Specification	Qty.	Purpose
13	Camera mounts and cable routing	Fixed overhead mount and two wrist mounts	1 set	Preserves camera identity, pose, and cable clearance
14	Training workstation	NVIDIA RTX A6000, 48 GB; physical GPU 1 used	1	Joint $\pi_{0.5}$ fine-tuning and validation
15	Network and ROS 2 bridge host	Project-configured communication and packet-assembly system	1 set	Connects robots, grippers, cameras, recorder, and policy
16	LeRobot software environment	LeRobot 0.5.2 with local changes; Dataset v3.0	1 config.	Recording, preprocessing, training, and deployment
17	Can and handled ring	Task 1 objects	1 set	Repositioning, grasp, and ring placement
18	Socket box and plug	Electrically unpowered insertion task fixture and matching plug	1 set	Repositioning and constrained insertion
19	Motherboard, memory module, and tray	Task 3 objects; yellow destination tray	1 set	Stabilization, extraction, transport, and placement
20	Task fixtures and reset references	Printed or manually assembled positioning aids	3 sets	Repeatable task initialization and terminal-state inspection
21	Safety equipment	Physical emergency stop, manufacturer protective functions, supervised operation	1 set	Deterministic stop and operator protection
22	Data storage	Storage for 119,650 frames, three-view video, checkpoints, and logs	1 set	Dataset and experiment traceability

Acknowledgements

We would like to express our sincere gratitude to our mentor, Prof. Yutong Ban, for his guidance throughout this project. His advice on research direction, robotic-system integration, experimental design, and technical presentation was essential to the development of the dual-arm manipulation platform and the completion of this thesis. His careful feedback encouraged us to examine the system objectively, distinguish demonstrated results from unsupported claims, and continuously improve both the engineering work and its documentation.

We are also deeply grateful to our instructor, Prof. Peisen Huang, for his guidance during the design-review process. His instruction on translating customer requirements into measurable engineering specifications, evaluating design alternatives, validating prototype performance, and communicating engineering decisions provided the methodological foundation for this project.

We sincerely thank SJTU SIRIUS Lab for providing the experimental platform, equipment, computing resources, and technical environment required for this work. We are especially grateful to Han Fang for his patient assistance and valuable guidance throughout the development, integration, and experimental validation of the system. His support helped us address practical difficulties encountered during the project and contributed significantly to the successful completion of the experiments.

We thank the faculty and staff of Global College, Shanghai Jiao Tong University, for providing the academic environment and organizational support necessary for this work. We also appreciate the Student Innovation Center for providing fabrication facilities and assistance with the custom grippers, fixtures, and other prototype components.

We further acknowledge the developers and research communities behind OpenPI, LeRobot, GELLO, ROS 2, PyTorch, and the related open-source tools used in this project. Their publicly available software, documentation, and research made it possible to construct, train, and deploy the integrated $\pi_{0.5}$ system within the project schedule.

Finally, we would like to thank our families and friends for their patience, encouragement, and support throughout the project. Their understanding sustained us during the intensive

periods of system integration, data collection, model training, physical validation, and thesis preparation.

A VISION-LANGUAGE-ACTION MODEL FOR DUAL-ARM ROBOTS IN 3C INDUSTRIAL PRODUCTION LINES

Background and objectives. Modern computer, communication, and consumer-electronics manufacturing must handle small components, frequent product changes, and contact-rich operations. Conventional fixed-program automation performs well in stable high-volume production, but each new component or assembly sequence can require substantial reprogramming, fixture redesign, and system integration. This project investigated whether a learning-based, language-conditioned dual-arm system could provide a more flexible foundation for representative 3C manipulation tasks. The objective was not to claim unrestricted factory autonomy, but to establish and validate an end-to-end engineering pipeline that connects customer requirements, hardware selection, teleoperated demonstration collection, multi-view sensing, model training, real-robot deployment, and quantitative physical evaluation.

The final task scope consisted of three bimanual operations. In the ring task, the left arm repositioned a can while the right arm grasped a handled black ring, placed it over the can, and released it. In the socket task, the right arm pushed a box containing a two-hole socket to the middle of the workspace while the left arm grasped a two-pin plug and inserted it. In the memory-module task, the left arm pulled and stabilized a motherboard while the right arm removed the memory module and placed it in a yellow tray. These tasks were selected to cover repositioning, grasping, coordinated bilateral roles, constrained insertion, extraction, transport, and placement.

Requirements and concept selection. Customer requirements were translated into six engineering specifications through quality function deployment. The design-stage targets were planning success above 90%, task-execution success above 70%, component-detection accuracy above 90%, pose-estimation error below 20 mm, failure-detection rate above 90%, and recovery success above 70%. The final evaluation measured planning, component detection, pose error, failure detection, and recovery in separate 25-experiment protocols and measured end-to-end task execution over 75 counted trials. The denominators and acceptance events were kept separate.

The concept-generation process separated the system into robot platform, teleoperation,

data collection, end-effector, and policy modules. Alternatives were screened for safety, interface compatibility, availability, manufacturability, and schedule feasibility, and the surviving candidates were evaluated using QFD-derived criteria. The selected physical system uses two Franka Research 3 robots, two GELLO leader devices, one overhead camera, two wrist cameras, and custom compliant grippers with TPU fingers. ACT and Diffusion Policy were retained as earlier policy baselines. As the project progressed beyond the DR3 stage, the final policy selection was updated to the LeRobot implementation of $\pi_{0.5}$ so that one jointly trained checkpoint could accept language instructions and generate bilateral actions for all three tasks.

Final system design. The workcell contains two upright FR3 arms operating in a shared tabletop region. Each arm contributes seven joint positions and one gripper value, producing a 16-dimensional bilateral proprioceptive state. The action interface uses the corresponding 16-dimensional ordering: seven absolute left-arm joint targets, left-gripper width, seven absolute right-arm joint targets, and right-gripper width. One fixed Intel RealSense D435i camera provides an overhead view of the shared scene, while two Intel RealSense D405 cameras provide close-range left- and right-wrist views. The three RGB streams provide complementary global and local observations near grasp, contact, insertion, extraction, and placement regions.

Demonstrations were collected bilaterally with two GELLO leaders. The operator controlled the left and right follower arms while the recording system stored synchronized visual observations, measured robot and gripper state, absolute action targets, task identity, and episode metadata. The same task instruction used during collection was retained for training and deployment. Before collection, the system checked leader zero positions, channel directions, gripper mappings, camera identities, packet freshness, and command limits. Episodes were accepted only when all three image streams and the complete numeric state/action record were present and the intended task was completed correctly.

Dataset and model training. The final dataset contains 240 bilateral episodes: 80 for the ring task, 80 for the socket task, and 80 for the memory-module task. In total, the dataset contains 119,650 frames. Eight episodes from each task were assigned to validation using the same fixed episode indices, leaving 216 episodes for training and 24 for validation. The split was performed at the episode level to prevent neighbouring frames from the same physical

trajectory from appearing in both sets. During training, the three task datasets were selected with equal probability so that the larger ring dataset did not dominate the optimization solely because it contained more frames.

The final controller was initialized from the official `pi05_base` model and trained through LeRobot 0.5.2 with project-specific modifications. The model received three resized and normalized RGB images, a language instruction, and the current bilateral state. The valid 16-dimensional state and action vectors were padded to the model-side representation while preserving their physical ordering. Joint training ran for 100,000 optimizer steps on an NVIDIA RTX A6000 with batch size 4 and AdamW optimization. Validation was performed every 5,000 steps. For checkpoint selection, the normalized flow-matching losses of the three tasks were averaged without frame-count weighting, giving every task equal influence. The minimum task-balanced macro loss, 0.036552, occurred at 65,000 steps, and this checkpoint was selected for physical deployment.

Deployment and safety boundary. Physical deployment used the same unified 65,000-step checkpoint for all three instructions. No task-specific model file was loaded. The policy received the selected instruction, the three current RGB images, and the 16-dimensional bilateral state. It predicted a 50-step continuous bimanual action chunk, and the executor consumed the first ten actions before acquiring an updated observation and requesting another prediction. At the configured 5 Hz action cadence, ten executed steps corresponded to a nominal two-second action segment.

The learned policy was not treated as the safety controller. Before enabling motion, a dry run verified the checkpoint identity, camera ordering and freshness, state/action dimensions, normalization statistics, instruction, network communication, and finite model output. Robot-side checks rejected invalid or excessive targets and enforced joint, gripper, workspace, timeout, protective-stop, emergency-stop, and manual-stop conditions. The socket fixture remained electrically unpowered. A counted trial received no manual correction after autonomous execution began.

Physical validation results. The selected checkpoint was evaluated on an NVIDIA GeForce RTX 5070 Ti in 75 counted physical trials, with 25 trials per task. Before each trial, the objects were reset and randomly perturbed around the nominal arrangement. The permitted variation was approximately 5 cm for the ring and memory-module scenes and approximately

3 cm for the socket scene. A trial was counted as successful only when the complete task-specific terminal condition was achieved. Partial completion was retained for failure analysis but was not counted as an end-to-end success.

The memory-module task succeeded in all 25 trials, producing an observed success rate of 100%. The ring task succeeded in 21 of 25 trials, producing 84%. The plug-insertion task succeeded in 13 of 25 trials, producing 52%. Across all tasks, the system completed 59 of 75 trials successfully, giving an overall observed success rate of 78.7%. The overall value exceeded the 70% task-execution target, and the memory-module and ring tasks exceeded the target individually. However, the socket task did not meet the target, demonstrating that the aggregate result did not represent uniform performance across all three operations.

The separate specification-level tests produced planning success of 23/25 (92%), component detection of 24/25 (96%), failure detection of 23/25 (92%), and recovery of 23/25 (92%). The recorded pose error remained below 20 mm under the stated rule that successful executions were not assigned an artificial zero error. These project-level measurements met the ES1, ES3, ES4, ES5, and ES6 thresholds. Because the rate-based tests contain only 25 observations each and the ES4 raw values were not supplied, the results support the project acceptance decisions but not precise population-level rate or pose-error distributions.

Failure analysis and discussion. The memory-module result indicates that the selected arm roles, camera observations, compliant grippers, demonstration data, and unified checkpoint were sufficient for the evaluated motherboard configuration. Nevertheless, 25 successful trials do not establish a zero failure probability or guarantee robustness to different motherboard poses, memory-module types, connector forces, lighting conditions, or camera displacement.

Three of the four ring failures occurred before secure transport. In two trials, the right gripper approached away from the handle and missed the grasp; in one trial, the ring base obstructed the handle approach. The remaining failure occurred during placement over the can. The dominant limitation was therefore handle acquisition rather than the initial can repositioning or the overall bilateral sequence. Additional demonstrations around displaced handle poses, improved wrist-camera visibility, a more repeatable pre-grasp approach, or a revised finger profile could directly address this failure pattern.

The socket task exposed the most important remaining performance bottleneck. Two of its twelve failures occurred because the left gripper did not acquire or retain the plug.

The other ten failures occurred after successful acquisition because the two pins were not aligned accurately enough with the two holes. The gross bimanual sequence was therefore feasible, but local contact precision was insufficient for reliable insertion. Possible contributors include residual socket-box pose variation after pushing, plug-pose variation inside the compliant gripper, limited close-range visual information, and the interval over which ten predicted actions were executed before replanning. These explanations are engineering hypotheses rather than measured causal conclusions because contact force, insertion depth, exact trial-level pose, and pre-contact action traces were not recorded.

The evaluation also has broader limitations. Initial object coordinates and their exact sampling distributions were not logged. Task duration was approximately one minute, but per-trial timing and end-to-end inference latency were not measured. The three visually distinct scenes were always paired with their corresponding instructions, so the experiment did not isolate sensitivity to language wording or distinguish language grounding from scene recognition. No physical ACT, task-specific, or alternative-checkpoint baseline was included; consequently, the results demonstrate successful $\pi_{0.5}$ deployment but do not establish its superiority over the DR3 controller.

Conclusions and recommendations. The project produced a functioning and traceable proof-of-concept system for language-conditioned bimanual manipulation. Its principal contribution is the integration of bilateral teleoperation, synchronized multi-view sensing, reproducible dataset construction, joint multi-task $\pi_{0.5}$ training, a unified 16-dimensional action interface, deterministic robot-side safeguards, and real-robot validation. The same checkpoint successfully executed all three scoped tasks, confirming the feasibility of unified multi-task deployment, while the task-dependent success rates showed that a shared representation does not eliminate the need for task-specific contact robustness.

Future development should first improve plug alignment and insertion through a more constrained fixture or passive lead-in geometry, better close-range visibility, more repeatable plug grasping, shorter execution between observations, and force- or torque-informed contact handling. The ring task would benefit from targeted demonstrations and pre-grasp alignment around difficult handle configurations. Later experiments should record exact initial coordinates and timing, test instruction paraphrases and mismatches, compare against ACT and task-specific baselines, and evaluate explicit task-level failure detection and bounded recov-

ery. These improvements would preserve the demonstrated unified architecture while addressing the precision, generalization, and validation limitations identified by the final physical trials.